



République Tunisienne



Ministère de l'Enseignement Supérieur et de la
Recherche Scientifique

Université de Tunis



ECOLE NATIONALE SUPÉRIEURE D'INGÉNIEURS DE TUNIS

Matière : TPM & Maintenance Intelligente

Analyse FMDS et Maintenance Prédictive

Application au Dataset MetroPT-3

Compresseur d'Air | Unité de Production d'Air (APU) | Métro de Porto

Présenté Par :

Manai Arbia
&
Lahbib Mayssa

Année Universitaire 2025–2026

Résumé

Ce projet porte sur l'analyse FMDS (Fiabilité, Maintenabilité, Disponibilité, Sécurité) et le développement d'un système de maintenance prédictive appliqué au dataset MetroPT-3 (Metropolitan Public Transport — Porto), issu du métro de Porto (Metro do Porto S.A.). Le dataset contient 1 516 948 enregistrements collectés à 1 Hz entre février et août 2020 sur le compresseur d'air de l'Unité de Production d'Air (APU) d'une rame ferroviaire.

L'objectif principal est d'estimer la durée de vie résiduelle (RUL — Remaining Useful Life) du compresseur à partir des données de 15 capteurs (7 analogiques, 8 numériques), en combinant l'analyse statistique FMDS, des modèles de classification supervisée (SVM, Arbre de Décision, Random Forest) pour le diagnostic des pannes, et des modèles de régression (Régression Linéaire, Random Forest) pour le pronostic RUL. Les quatre pannes documentées sont toutes de type fuite d'air.

Les résultats de l'analyse FMDS révèlent une disponibilité de 96,89 %, un MTBF de 681,17 heures (environ 28 jours entre deux pannes) et un MTTR de 21,87 heures. Les modèles de diagnostic atteignent des performances excellentes : le Random Forest obtient une précision de 99,45 % et un AUC-ROC de 0,9998. Pour le pronostic RUL, le Random Forest atteint un R^2 de 0,796 et un RMSE de 225,87 heures, confirmant la faisabilité d'une maintenance prédictive opérationnelle sur ce type d'équipement ferroviaire.

Une modélisation Weibull ($\beta = 1,714$ | $\eta = 757,5$ h) complète l'analyse FMDS et confirme une usure progressive du compresseur avec un écart inférieur à 1 % entre les deux approches.

Mots-clés : FMDS, Maintenance prédictive, RUL, Machine Learning, Random Forest, SVM, Compresseur d'air, MetroPT-3, Détection de pannes, Pronostic, Loi de Weibull, Fiabilité.

Table des matières

RESUME	1
INTRODUCTION GENERALE	1
1. INTRODUCTION ET CONTEXTE INDUSTRIEL.....	1
2. OBJECTIFS DU PROJET ET PROBLEMATIQUE	1
2.1 PROBLEMATIQUE.....	1
2.2 OBJECTIFS.....	1
2.3 PERIMETRE	1
3. PRESENTATION DU SYSTEME ÉTUDIÉ	2
3.1 ARCHITECTURE DE L'APU	2
3.2 DEFAILLANCES DOCUMENTÉES.....	2
4. DONNEES UTILISEES ET PRETRAITEMENTS.....	3
4.1 LE DATASET METROPT-3.....	3
4.2 DESCRIPTION DES VARIABLES.....	5
4.3 PRETRAITEMENTS APPLIQUES	6
5. ANALYSE FMDS	6
5.1 DEFINITION DES INDICATEURS.....	6
5.2 RESULTATS DES INDICATEURS FMDS.....	7
5.3 INTERPRETATION	8
5.4 MODELISATION DE LA FIABILITE — LOI DE WEIBULL	8
5.4.1 PRESENTATION DU MODELE	8
5.4.2 FORMULES MATHÉMATIQUES	9
5.4.3 DONNEES UTILISEES.....	9
5.4.4 RESULTATS DE L'AJUSTEMENT WEIBULL.....	9
5.4.5 INTERPRETATION DES RESULTATS	9
5.4.6 VISUALISATION — COURBES DE FIABILITE WEIBULL.....	10

6. DIAGNOSTIC ET PRONOSTIC	11
6.1 <i>DIAGNOSTIC — ANALYSE DES SIGNAUX CAPTEURS</i>	11
6.1.1 <i>REPARTITION NORMALE / PANNE</i>	11
6.1.2 <i>COMPORTEMENT DES CAPTEURS</i>	11
6.1.3 <i>MODELES DE CLASSIFICATION</i>	12
6.2 <i>PRONOSTIC — ESTIMATION DU RUL (REMAINING USEFUL LIFE)</i>	13
6.2.1 <i>MODELES DE PRONOSTIC</i>	13
6.2.2 <i>RESULTATS DU PRONOSTIC RUL</i>	14
 7. VALIDATION DES MODELES	15
7.1 <i>METRIQUES DE CLASSIFICATION</i>	15
7.2 <i>MATRICES DE CONFUSION</i>	16
7.3 <i>COURBES ROC</i>	17
7.4 <i>COURBES D'APPRENTISSAGE</i>	18
7.5 <i>IMPORTANCE DES VARIABLES — RANDOM FOREST</i>	18
 8. SYNTHESE, RECOMMANDATIONS ET PERSPECTIVES	19
8.1 <i>SYNTHESE GENERALE DES RESULTATS</i>	19
8.2 <i>RECOMMANDATIONS PRATIQUES</i>	20
8.3 <i>LIMITES DES APPROCHES</i>	21
8.4 <i>PERSPECTIVES D'AMELIORATION</i>	21
 CONCLUSION GENERALE	23
 9. BIBLIOGRAPHIE ET ANNEXES	24
9.1 <i>REFERENCES BIBLIOGRAPHIQUES</i>	24
9.2 <i>ANNEXES A : CODE DE REALISATION</i>	24

Table des figures

FIGURE 1: PANNES DOCUMENTEES ET CALCUL DES INDICATEURS FMDS (METROPT-3)	2
FIGURE 2: STRUCTURE ET CHARGEMENT DU DATASET METROPT-3	3
FIGURE 3: STATISTIQUES DESCRIPTIVES DES VARIABLES DU DATASET METROPT-3	5
FIGURE 4: RESUME DE LA PREPARATION DES DONNEES POUR LE DIAGNOSTIC.....	6
FIGURE 5: SYNTHESE GRAPHIQUE DE L'ANALYSE FMDS (METROPT-3, FEV-AOUT 2020)	7
FIGURE 6: DUREE DE REPARATION PAR PANNE (TTR vs MTTR = 21,87 H)	8
FIGURE 7: TAUX DE DISPONIBILITE GLOBAL DU COMPRESSEUR — DASHBOARD FMDS	8
FIGURE 8— ANALYSE DE FIABILITE WEIBULL — COMPRESSEUR METROPT-3 ($\beta=1,714$ $H=757,5H$)	10
FIGURE 9: REPARTITION DES ETATS NORMAL / PANNE	11
FIGURE 10: ÉVOLUTION TEMPORELLE DES 7 CAPTEURS ANALOGIQUES AVEC ZONES DE PANNES IDENTIFIEES.....	12
FIGURE 11 : RESULTATS DES TROIS MODELES DE CLASSIFICATION SUPERVISEE (SVM, ARBRE DE DECISION, RANDOM FOREST) SUR LE DATASET EQUILIBRE (11 939 INSTANCES DE TEST).....	12
FIGURE 12: CALCUL DU RUL ET PARAMETRES D'ENTRAINEMENT DES MODELES DE PRONOSTIC	13
FIGURE 13: PRONOSTIC RUL : PREDICTIONS DES MODELES, EVOLUTION TEMPORELLE ET COMPARAISON DES PERFORMANCES.....	14
FIGURE 14: TABLEAU DE BORD DE VALIDATION : METRIQUES DE CLASSIFICATION (HEATMAP) ET REGRESSION RUL	15
FIGURE 15 : MATRICES DE CONFUSION ET SCORES DES TROIS MODELES DE CLASSIFICATION	16
FIGURE 16: COURBES ROC DES TROIS MODELES DE CLASSIFICATION	17
FIGURE 17: COURBES D'APPRENTISSAGE DES TROIS MODELES (SVM, ARBRE DE DECISION, RANDOM FOREST).....	18
FIGURE 18: IMPORTANCE DES FEATURES — RANDOM FOREST	18
FIGURE 19: SYNTHESE GENERALE DU PROJET : INDICATEURS FMDS, PERFORMANCES DES MODELES, LIMITES ET PERSPECTIVES	20
FIGURE 20: LIMITES DES APPROCHES UTILISEES.....	21
FIGURE 21: PERSPECTIVES D'AMELIORATION A COURT, MOYEN ET LONG TERME.....	21

Liste des tableaux

TABLE 1— OBJECTIFS DU PROJET	1
TABLE 2— SOUS-SYSTEMES ET CAPTEURS DE L'APU.....	2
TABLE 3— CARACTERISTIQUES GENERALES DU DATASET METROPT-3.....	3
TABLE 4— DESCRIPTION DES VARIABLES (CAPTEURS ANALOGIQUES ET NUMERIQUES)	5
TABLE 5— DEFINITION DES INDICATEURS FMDS.....	6
TABLE 6— RESULTATS DES INDICATEURS FMDS (Fev–AOUT 2020)	7
TABLE 7— INTERVALLES DE FONCTIONNEMENT UTILISES POUR L'AJUSTEMENT WEIBULL	9
TABLE 8— PARAMETRES WEIBULL ESTIMES PAR MLE — COMPRESSEUR METROPT-3	9
TABLE 9— REPARTITION DES ETATS NORMAL / PANNE DANS LE DATASET	11
TABLE 10— COMPORTEMENT DES CAPTEURS EN ETAT NORMAL ET EN PANNE.....	11
TABLE 11— MODELES DE PRONOSTIC RUL : PRINCIPES ET COMPARAISON	13
TABLE 12— RESULTATS DU PRONOSTIC RUL (REGRESSION LINEAIRE VS RANDOM FOREST)	14
TABLE 13— METRIQUES DE CLASSIFICATION DES TROIS MODELES	15
TABLE 14— SCORES AUC-ROC DES TROIS MODELES DE CLASSIFICATION.....	17
TABLE 15— IMPORTANCE DES VARIABLES SELON LE MODELE RANDOM FOREST.....	19
TABLE 16— SYNTHESE GENERALE DES RESULTATS DU PROJET.....	19

Introduction Générale

Face aux exigences de fiabilité dans le transport ferroviaire, la maintenance prédictive s'impose comme une alternative efficace à la maintenance corrective. Ce projet applique cette approche au compresseur d'air de l'APU du métro de Porto, en exploitant le dataset réel MetroPT-3 (1 516 948 enregistrements, 15 capteurs, février–août 2020).

L'objectif est triple : analyser la fiabilité par les indicateurs FMDS et la loi de Weibull, détecter automatiquement les pannes par classification supervisée (SVM, Arbre de Décision, Random Forest), et estimer la durée de vie résiduelle (RUL) par régression.

Ce rapport est organisé en neuf sections, couvrant la présentation du système, les prétraitements, les résultats FMDS, le diagnostic, le pronostic, la validation des modèles et les recommandations pratiques.

1. Introduction et Contexte Industriel

Le compresseur d'air est un équipement critique dans le transport ferroviaire. Il alimente les systèmes pneumatiques essentiels à la sécurité : freinage, portes automatiques, suspensions et circuits de commande. Toute défaillance peut immobiliser la rame immédiatement.

Ce projet s'appuie sur des données réelles collectées sur le compresseur d'une rame du métro de Porto (Metro do Porto S.A.), entre février et août 2020, à une fréquence d'acquisition de 1 Hz. Ce jeu de données, appelé MetroPT-3, a été publié par l'Institut INESC TEC et déposé sur le référentiel UCI Machine Learning Repository.

Face aux contraintes opérationnelles, deux stratégies coexistent habituellement : la maintenance corrective (intervention après panne) et la maintenance préventive systématique (intervalles fixes). Ce projet explore une troisième voie — la maintenance prédictive — en exploitant les données capteurs pour anticiper les défaillances.

Une modélisation probabiliste par la loi de Weibull vient compléter cette analyse pour valider mathématiquement les indicateurs de fiabilité obtenus.

2. Objectifs du Projet et Problématique

2.1 Problématique

Le compresseur d'air de l'APU tombe en panne plusieurs fois par an, toujours pour la même cause : une fuite d'air. Chaque panne nécessite une intervention de plusieurs heures. L'objectif est de comprendre et d'anticiper ces défaillances à partir des données capteurs.

2.2 Objectifs

Le projet vise à développer une approche complète de maintenance prédictive pour le compresseur d'air APU du métro de Porto, en combinant l'analyse de fiabilité FMDS, le diagnostic par classification supervisée et le pronostic par estimation du RUL. Le tableau ci-dessous synthétise les trois axes d'objectifs retenus.

Table 1— Objectifs du projet

Objectif Analytique	Objectif Technique	Objectif Opérationnel
Analyser le comportement des capteurs par analyse descriptive et FMDS.	Mettre en œuvre un prétraitement adapté aux séries temporelles industrielles.	Fournir des indicateurs de fiabilité et des modèles de détection de pannes exploitables.

Ces trois objectifs sont complémentaires : l'analyse FMDS établit le diagnostic de fiabilité du système, les modèles de classification permettent la détection automatique des pannes en temps réel, et le pronostic RUL anticipe les défaillances pour planifier les interventions de maintenance de façon optimisée.

2.3 Périmètre

Ce projet couvre l'analyse FMDS, le prétraitement, le diagnostic par classification et le pronostic par estimation du RUL (Remaining Useful Life). Les quatre pannes documentées sont toutes de type fuite d'air.

3. Présentation du Système Étudié

3.1 Architecture de l'APU

L'Unité de Production d'Air (APU) est composée de quatre sous-systèmes interdépendants :

Table 2— Sous-systèmes et capteurs de l'APU

Sous-système	Fonction	Capteurs associés
Circuit de compression	Comprimer l'air ambiant	TP2, TP3, H1, Motor_current, COMP, DV_electric, MPG
Circuit de séchage	Éliminer l'humidité de l'air	DV_pressure, Pressure_switch, TOWERS
Circuit de lubrification	Lubrifier le compresseur	Oil_temperature, Oil_level
Circuit de stockage/distribution	Stocker et distribuer l'air comprimé	Reservoirs, LPS, Caudal_impulse

3.2 Défaillances Documentées

Durant la période d'enregistrement, quatre pannes de type fuite d'air ont été rapportées :

● TABLEAU DES 4 PANNES DOCUMENTÉES – RAPPORT OFFICIEL UCI METROPT-3

#	DÉBUT PANNE	FIN PANNE	TTR (HEURES)	TBF SUIVANT (H)	TYPE	SÉVÉRITÉ
01	18/04/2020 00:00	18/04/2020 23:59	24.0 h	984 h	Air Leak	High Stress
02	29/05/2020 23:30	30/05/2020 06:00	6.50 h	148 h	Air Leak	High Stress
03	05/06/2020 10:00	07/06/2020 14:30	52.50 h	912 h	Air Leak	Critique
04	15/07/2020 14:30	15/07/2020 19:00	4.50 h	—	Air Leak	Modéré

Figure 1: Pannes documentées et calcul des indicateurs FMDS (MetroPT-3)

Le tableau ci-après récapitule les quatre événements de panne enregistrés durant la période d'observation, avec leurs dates, durées et types, tels que rapportés par l'exploitant.

Ce tableau met en évidence la grande disparité entre les durées de panne : de 4,5 heures pour la panne n°4 à 52,5 heures pour la panne n°3, qui représente à elle seule 60 % du temps total en arrêt. Cette irrégularité souligne la difficulté de planifier des interventions sur la seule base d'un calendrier fixe, et justifie le recours à la maintenance prédictive.

La colonne 'TBF suivant' révèle une grande irrégularité entre les intervalles de fonctionnement : de 148 heures seulement entre P2 et P3, contre 984 heures entre P1 et P2. La colonne 'Sévérité' confirme que la panne n°3 est la plus critique, avec un niveau 'Critique' et 52,5 heures d'arrêt.

4. Données Utilisées et Prétraitements

4.1 Le Dataset MetroPT-3

Le tableau ci-dessous présente les caractéristiques générales du jeu de données MetroPT-3, collecté sur un compresseur d'air en conditions réelles d'exploitation au sein du métro de Porto.

Table 3— Caractéristiques générales du dataset MetroPT-3

Caractéristique	Valeur
Nombre d'enregistrements	1 516 948 points de données
Nombre de variables	15 (7 analogiques + 8 numériques)
Fréquence d'acquisition	1 Hz (une mesure par seconde)
Période couverte	Février – Août 2020 (≈ 6 mois)
Valeurs manquantes	Aucune
Pannes documentées	4 événements (fuites d'air)
Source	UCI ML Repository — DOI : 10.24432/C5VW3R

```

PROJET MAINTENANCE PRÉDICTIVE - MetroPT-3

✓ Fichier CSV trouvé : data\MetroPT3(AirCompressor).csv

➡ SECTIONS 1-4 : ANALYSE FMS

=====
ÉTAPE 1 - CHARGEMENT DU DATASET MetroPT-3
=====

✓ Fichier chargé avec succès
Fichier      : data\MetroPT3(AirCompressor).csv
Lignes x Colonnes : 1,516,948 x 16
Début       : 2020-02-01 00:00:00
Fin        : 2020-09-01 03:59:50
Durée totale : 213 jours

Colonnes disponibles :
['timestamp', 'TP2', 'TP3', 'H1', 'DV_pressure', 'Reservoirs', 'Oil_temperature', 'Motor_current', 'COMP', 'DV_electric', 'Towers', 'MPG', 'LPS', 'Pressure_switch', 'el', 'Caudal_impulses']

Types de données :
timestamp      : datetime64[ns]
TP2            : float64
TP3            : float64
H1             : float64
DV_pressure    : float64
Reservoirs     : float64
Oil_temperature : float64
Motor_current  : float64
COMP           : float64
DV_electric    : float64
Towers         : float64
MPG            : float64
LPS            : float64
Pressure_switch : float64
oil_level      : float64
Caudal_impulses : float64
  
```

Figure 2: Structure et chargement du dataset MetroPT-3

Ce tableau synthétise les informations clés du dataset : 1 516 948 points collectés à 1 Hz sur 6 mois, avec 15 variables (7 analogiques, 8 numériques) et aucune valeur manquante. La richesse de ce dataset — en termes de volume, de diversité des capteurs et de documentation des pannes — en fait un support idéal pour développer et évaluer des modèles de maintenance prédictive.

Pourquoi le dataset MetroPT-3 ?

Le dataset MetroPT-3 a été retenu pour ce projet pour plusieurs raisons fondamentales. Premièrement, il s'agit de données réelles collectées en conditions opérationnelles sur une rame du métro de Porto (Metro do Porto S.A.), et non de données simulées en laboratoire. Cela garantit que les défis rencontrés — bruit de mesure, déséquilibre de classes, irrégularité des pannes — reflètent fidèlement les problématiques industrielles réelles auxquelles un ingénieur de maintenance serait confronté.

Deuxièmement, le dataset couvre les trois grandes tâches de la maintenance prédictive : la détection d'anomalies (est-ce que le système est en panne ?), le diagnostic (quels capteurs sont affectés ?) et le pronostic (combien de temps reste-t-il avant la prochaine panne ?). Cette polyvalence en fait un support complet pour un projet d'ingénierie qui vise à explorer l'ensemble du pipeline de maintenance prédictive, depuis l'analyse FMDS jusqu'à l'estimation du RUL.

Troisièmement, le dataset est publiquement disponible sur l'UCI Machine Learning Repository (DOI : 10.24432/C5VW3R), ce qui assure sa reproductibilité et sa crédibilité scientifique. Il a été utilisé dans des travaux de recherche publiés (IEEE DSAA 2021, Scientific Data 2022), ce qui permet de comparer nos résultats à des références existantes dans la littérature.

Enfin, la fréquence d'acquisition de 1 Hz sur 6 mois génère plus de 1,5 million de points de données, ce qui constitue un volume suffisant pour entraîner et valider des modèles de machine learning robustes, tout en représentant un vrai défi de traitement des séries temporelles industrielles à haute fréquence.

Contenu et structure du dataset

Le dataset est structuré comme une série temporelle multivariée : chaque ligne correspond à un instant t (une seconde), et chaque colonne correspond à la valeur d'un capteur à cet instant. Il contient 15 variables réparties en deux groupes : 7 capteurs analogiques mesurant des grandeurs physiques continues (pressions en bar, courant en ampères, température en °C), et 8 capteurs numériques fournissant des signaux binaires (0 ou 1) indiquant l'état des actionneurs et des dispositifs de sécurité.

Les capteurs analogiques couvrent l'ensemble du cycle de compression : TP2 et TP3 mesurent la pression en entrée et en sortie du compresseur, H1 capture la chute de pression au niveau du séparateur cyclonique, DV_pressure surveille l'efficacité des sécheurs d'air, Réservoirs mesure la pression dans les réservoirs de stockage, Motor_current reflète la charge électrique du moteur, et Oil_temperature indique la température de l'huile de lubrification. Ces variables sont directement liées aux mécanismes physiques des fuites d'air.

Le dataset ne contient aucune valeur manquante, ce qui est rare pour des données industrielles réelles et simplifie considérablement le prétraitement. En revanche, il présente un fort déséquilibre de classes : les 4 pannes documentées représentent seulement 1,97 % des 1 516 948 instants enregistrés. Ce déséquilibre est typique des systèmes fiables mais il exige un rééquilibrage avant l'entraînement des modèles supervisés, sous peine d'obtenir des classifieurs biaisés incapables de détecter les rares instants de panne.

Lien avec la maintenance prédictive

Chaque variable du dataset joue un rôle précis dans la stratégie de maintenance prédictive. Les capteurs de pression (TP2, TP3, Réservoirs) permettent de détecter les anomalies de compression caractéristiques d'une fuite d'air. La variable DV_pressure révèle le comportement des sécheurs et s'avère — comme le confirmeront les résultats du Random Forest — le capteur le plus discriminant avec 27,27 % d'importance. Oil_temperature, quant à elle, constitue un précurseur thermique de la dégradation : une élévation anormale de la température de l'huile précède généralement les pannes, ce qui en fait un signal d'alerte précoce de premier ordre.

En résumé, le MetroPT-3 est un dataset à la fois réaliste, complet et bien documenté. Il permet d'aborder la maintenance prédictive sous ses trois angles : analyse de la fiabilité (FMDS), détection des pannes (diagnostic par classification) et estimation de la durée de vie résiduelle

(pronostic RUL). C'est précisément pour cette complémentarité qu'il a été choisi comme support de ce projet.

4.2 Description des Variables

Le tableau suivant décrit chacune des 15 variables du dataset, en distinguant les capteurs analogiques (mesures physiques continues) des capteurs numériques (signaux d'état binaires).

Table 4— Description des variables (capteurs analogiques et numériques)

Variable	Unité	Description	Type
TP2	bar	Pression sur le compresseur	Analogique
TP3	bar	Pression au panneau pneumatique	Analogique
H1	bar	Chute de pression — séparateur cyclonique	Analogique
DV_pressure	bar	Chute de pression aux sécheurs	Analogique
Reservoirs	bar	Pression aval des réservoirs	Analogique
Motor_current	A	Courant moteur (0 / 4 / 7–9 A)	Analogique
Oil_temperature	°C	Température de l'huile	Analogique
COMP	0/1	Vanne admission d'air	Numérique
DV_electric	0/1	Vanne sortie compresseur	Numérique
TOWERS	0/1	Signaux d'état binaires des actionneurs (8 variables)	Numérique
MPG	0/1	Démarrage sous charge	Numérique
LPS	0/1	Alarme basse pression	Numérique
Pressure_sw	0/1	Décharge sécheurs	Numérique
Oil_level	0/1	Alarme niveau huile	Numérique
Caudal_imp	0/1	Débit air réservoirs	Numérique

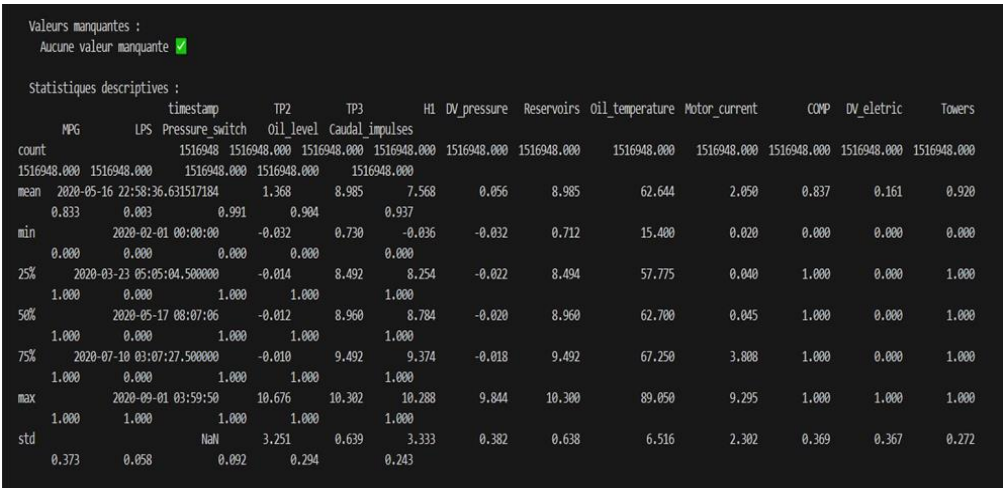


Figure 3: statistiques descriptives des variables du dataset metropt-3

L'analyse des variables révèle que les capteurs analogiques tels que TP2, TP3 et Reservoirs fournissent des mesures de pression essentielles pour détecter les fuites d'air. La variable DV_pressure et Oil_temperature s'avèrent particulièrement discriminantes, car elles reflètent directement les anomalies dans les circuits de séchage et de lubrification. Les 8 variables numériques complètent ce tableau en indiquant l'état des actionneurs et des capteurs de seuil, permettant ainsi une surveillance complète du cycle de compression.

4.3 Prétraitements Appliqués

a) Normalisation centrée-réduite : Les données ont été normalisées à l'aide de la méthode StandardScaler (normalisation centrée-réduite). Cette technique consiste à transformer les variables pour avoir une moyenne égale à 0 et un écart-type égal à 1.

Cette normalisation est particulièrement adaptée aux modèles comme SVM et les modèles linéaires, car elle améliore la convergence et les performances.

b) Construction de l'indicateur de panne : Variable binaire créée à partir des rapports de l'exploitant : 0 = normal, 1 = panne.

c) Rééquilibrage des classes : 29 954 instants de panne + 29 740 instants normaux = 59 694 échantillons équilibrés. Découpage 80/20 en ordre chronologique.

```
=====
DIAGNOSTIC – Analyse des signaux capteurs
=====
✅ Graphique sauvegardé : outputs\diagnostic_signaux.png

=====
DIAGNOSTIC – Préparation des données
=====
Features utilisées : ['TP2', 'TP3', 'H1', 'DV_pressure', 'Reservoirs', 'oil_temperature', 'Motor_current']
Échantillons totaux : 59,694
Ratio panne/normal : 29954/29740
Train : 47,755 | Test : 11,939
```

Figure 4: Résumé de la préparation des données pour le diagnostic

La figure ci-dessous présente le pipeline complet de préparation des données, depuis la normalisation jusqu'à la constitution des ensembles d'entraînement et de test.

Le prétraitement appliqué garantit des données exploitables pour les modèles supervisés. La normalisation centrée-réduite (Z-score) évite que les variables à grande amplitude dominent l'apprentissage, tandis que le rééquilibrage des classes ($\approx 30\,000$ instants de panne contre $1\,487\,000$ normaux) est indispensable pour que les classifieurs ne soient pas biaisés vers l'état normal.

5. Analyse FMDS

5.1 Définition des Indicateurs

Le tableau suivant définit les quatre indicateurs FMDS utilisés pour évaluer les performances du compresseur d'air sur la période d'observation.

Table 5— Définition des indicateurs FMDS

Indicateur	Symbole	Définition
Temps Moyen Entre Pannes	MTBF	Durée moyenne de bon fonctionnement entre deux pannes.
Temps Moyen de Réparation	MTTR	Durée moyenne d'une intervention de maintenance.
Taux de défaillance	λ	Nombre moyen de pannes par heure.
Disponibilité	A	Pourcentage de temps où le système est opérationnel.

Ces quatre indicateurs sont complémentaires : le MTBF et le taux de défaillance λ caractérisent la fiabilité du système (fréquence des pannes), le MTTR mesure sa maintenabilité (rapidité de remise en service), et la disponibilité A synthétise les deux en un indicateur global d'efficacité opérationnelle. Ensemble, ils permettent d'identifier les axes d'amélioration prioritaires et justifient le recours à la maintenance prédictive pour réduire à la fois la fréquence et la durée des arrêts.

5.2 Résultats des Indicateurs FMDS

L'analyse FMDS a été conduite sur la période d'observation complète allant du 1er février au 1er septembre 2020, soit 213 jours (5 116 heures). Les quatre pannes documentées sont toutes de type fuite d'air. Le tableau ci-dessous présente les résultats calculés pour chacun des indicateurs FMDS.

Table 6— Résultats des indicateurs FMDS (Fév–Août 2020)

Indicateur	Valeur calculée	Interprétation
Temps en fonctionnement	5 028,51 h	Durée effective de bon fonctionnement
Temps total en panne (TTR)	87,48 h	Cumul des 4 pannes
MTBF	681,17 h	≈ 28,4 jours entre deux pannes
MTTR	21,87 h	≈ 0,9 jour par intervention
Taux de défaillance λ	0,001468 /h	≈ 1,06 panne/mois
Disponibilité A	96,89 %	Niveau BON (seuil recommandé : 95–99 %)



Figure 5: Synthèse graphique de l'analyse FMDS (MetroPT-3, Fév–Août 2020)

La figure suivante présente une synthèse graphique des indicateurs FMDS calculés sur la période de février à août 2020.

La synthèse graphique confirme les bons résultats de disponibilité (96,89 %) et un MTBF de 681 heures (environ 28 jours). Cependant, le MTTR de 21,87 heures, fortement influencé par la panne n°3 (52,5 h), révèle que la durée des réparations reste le principal levier d'amélioration. Ces indicateurs servent de référence pour mesurer l'impact futur des modèles prédictifs.

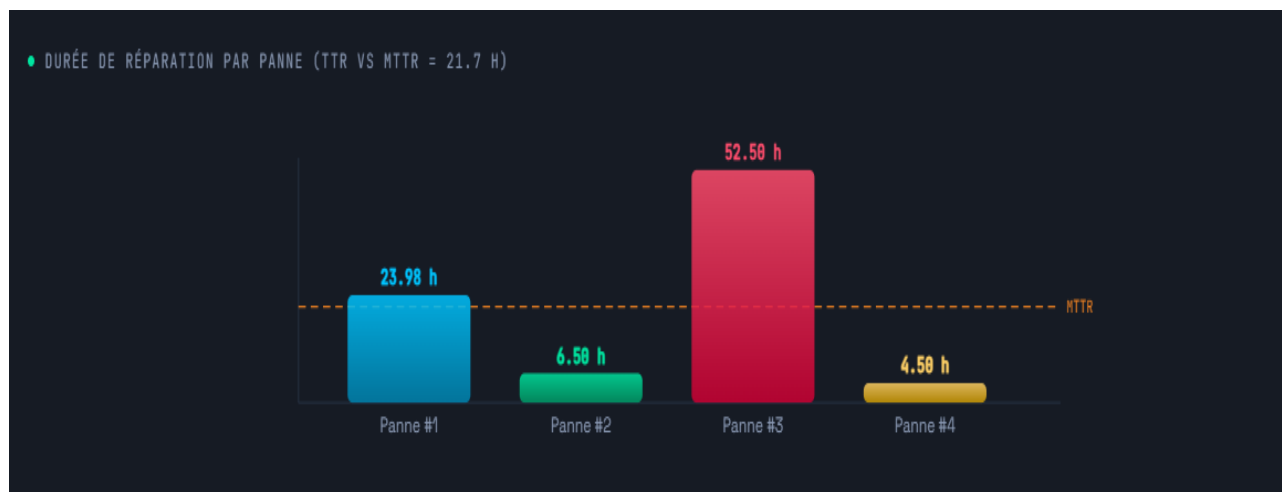


Figure 6: Durée de réparation par panne (TTR vs MTTR = 21,87 h)

Le graphique ci-dessus illustre la durée de réparation de chaque panne comparée au MTTR moyen (21,87 h, ligne pointillée orange). La panne n°3 se distingue nettement avec 52,50 heures d'arrêt — soit 2,4 fois le MTTR — confirmant son caractère exceptionnel et son impact dominant sur l'ensemble des indicateurs FMDS. Les pannes n°2 et n°4, inférieures au MTTR, témoignent d'interventions rapides et efficaces.

5.3 Interprétation

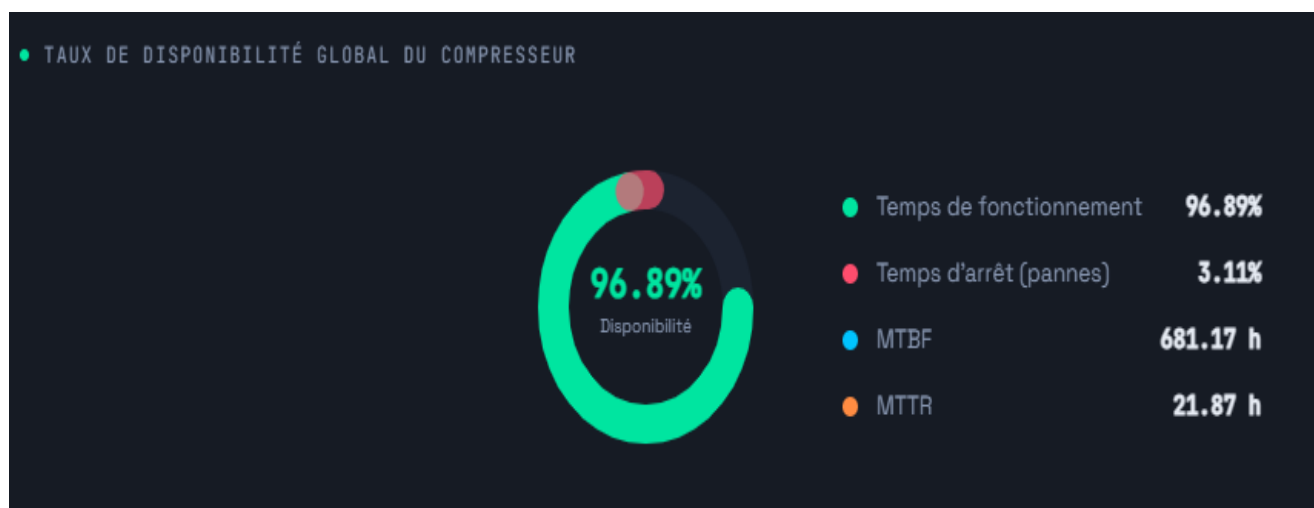


Figure 7: Taux de disponibilité global du compresseur — Dashboard FMDS

Le dashboard ci-dessus synthétise visuellement les principaux indicateurs FMDS du compresseur. Le taux de disponibilité global atteint 96,89 %, avec un temps de fonctionnement de 96,89 % contre 3,11 % de temps d'arrêt cumulé. Le MTBF de 681,17 h confirme une bonne fiabilité globale, tandis que le MTTR de 21,87 h reste le principal levier d'amélioration pour rapprocher la disponibilité du seuil industriel de 99 %.

- Ces résultats seront validés mathématiquement par la modélisation Weibull dans la section suivante.

5.4 Modélisation de la Fiabilité — Loi de Weibull

5.4.1 Présentation du Modèle

La loi de Weibull est un modèle probabiliste largement utilisé en fiabilité industrielle pour modéliser la durée de vie des équipements et analyser leur comportement en défaillance. Elle permet de caractériser trois phases distinctes du cycle de vie d'un système : la mortalité infantile ($\beta < 1$), les pannes aléatoires ($\beta \approx 1$) et l'usure progressive ($\beta > 1$).

Dans le cadre de ce projet, la loi de Weibull est appliquée aux temps de fonctionnement entre les quatre pannes documentées du compresseur MetroPT-3, afin de modéliser mathématiquement sa fiabilité et de valider les indicateurs FMDS calculés précédemment.

5.4.2 Formules Mathématiques

La loi de Weibull est définie par deux paramètres principaux :

- **β (bêta) : paramètre de forme** — caractérise le type de défaillance
- **η (êta) : paramètre d'échelle** — lié à la durée de vie caractéristique

Les fonctions fondamentales de la loi de Weibull sont :

$R(t) = \exp[-(t/\eta)^\beta]$	Fonction de Fiabilité
$F(t) = 1 - R(t)$	Fonction de Défaillance
$h(t) = (\beta/\eta) \times (t/\eta)^{(\beta-1)}$	Taux de Défaillance
$MTBF = \eta \times \Gamma(1 + 1/\beta)$	Espérance de vie

5.4.3 Données Utilisées

Le modèle de Weibull est ajusté sur les trois intervalles de fonctionnement observés entre les quatre pannes documentées de la période Février–Août 2020 :

Table 7— Intervalles de fonctionnement utilisés pour l'ajustement Weibull

Intervalle	Début	Fin	Durée (h)
P1 → P2	18 Avril 2020	29 Mai 2020	984 h
P2 → P3	30 Mai 2020	5 Juin 2020	148 h
P3 → P4	7 Juin 2020	15 Juil. 2020	912 h

5.4.4 Résultats de l'Ajustement Weibull

L'ajustement de la loi de Weibull par la méthode du Maximum de Vraisemblance (MLE), implémentée via la bibliothèque SciPy en Python, donne les paramètres suivants :

Table 8— Paramètres Weibull estimés par MLE — Compresseur MetroPT-3

Paramètre	Symbole	Valeur	Interprétation
Paramètre de forme	β (bêta)	1,714	Usure progressive ($\beta > 1$)
Paramètre d'échelle	η (êta)	757,5 h	Durée de vie caractéristique
MTBF Weibull	$E[T]$	675,5 h	$\approx$ MTBF calculé = 681,17 h ✓

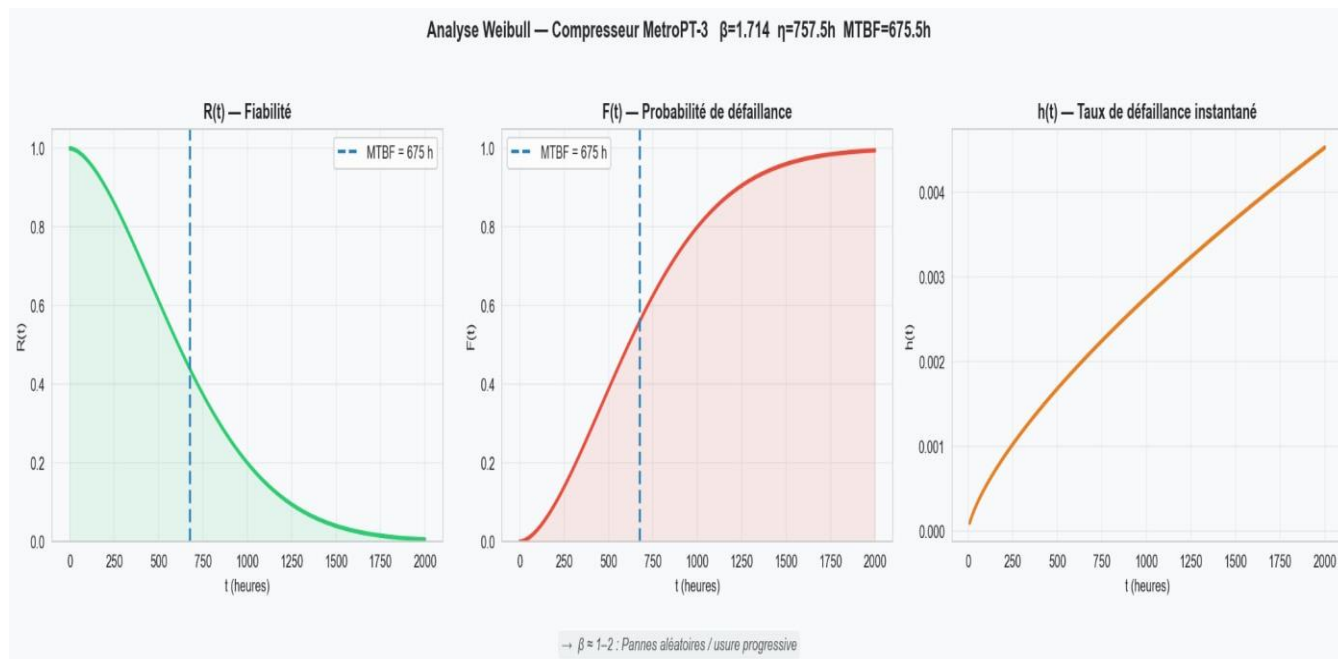
5.4.5 Interprétation des Résultats

Les résultats de l'ajustement Weibull révèlent trois informations essentielles :

- **$\beta = 1,714 > 1$** → Pannes aléatoires / usure progressive : le taux de défaillance augmente progressivement mais de façon modérée.
- **$\eta = 757,5$ h** → Durée de vie caractéristique : à $t = \eta$, environ 63,2 % des compresseurs similaires auraient subi une défaillance ($F(\eta) = 1 - e^{-1} \approx 63,2 \%$).
- **MTBF Weibull = 675,5 h $\approx$ MTBF calculé = 681,17 h** : l'écart est de seulement 5,67 heures (< 1 %), ce qui valide la cohérence des deux approches et confirme la fiabilité des indicateurs FMDS calculés dans la section 5.2.

5.4.6 Visualisation — Courbes de Fiabilité Weibull

La figure suivante présente les quatre courbes caractéristiques de la loi de Weibull ajustée sur les données du compresseur MetroPT-3 : la fonction de fiabilité $R(t)$, les paramètres estimés, la fonction de défaillance $F(t)$ et le taux de défaillance $h(t)$.



La courbe $R(t)$ confirme une décroissance progressive de la fiabilité avec le temps, atteignant **$R(681h) \approx 40\%$** au niveau du **MTBF** calculé. La courbe $h(t)$ montre une augmentation monotone du taux de défaillance, caractéristique de la phase d'usure ($\beta > 1$), ce qui justifie la mise en place d'une politique de maintenance prédictive avant l'atteinte de la durée de vie caractéristique **$\eta = 757,5$ heures**.

Conclusion FMDS : L'analyse FMDS du compresseur d'air MetroPT-3 révèle une disponibilité de 96,89 %, un MTBF de 681,17 h (≈ 28 jours) et un MTTR de 21,87 h. La modélisation Weibull ($\beta = 1,714$ | $\eta = 757,5$ h) confirme une usure progressive du système et valide ces indicateurs avec un écart inférieur à 1 % entre MTBF FMDS et MTBF Weibull. Une intervention préventive est recommandée avant $t = 600$ h .

6. Diagnostic et Pronostic

6.1 Diagnostic — Analyse des Signaux Capteurs

6.1.1 Répartition normale / panne

Le tableau suivant présente la répartition des instants normaux et des instants de panne dans le dataset original, mettant en évidence le fort déséquilibre de classes caractéristique des systèmes industriels fiables.

Table 9— Répartition des états normal / panne dans le dataset

État	Nombre d'instants	Proportion
Normal (0)	1 486 994	98,03 %
Panne (1)	29 954	1,97 %

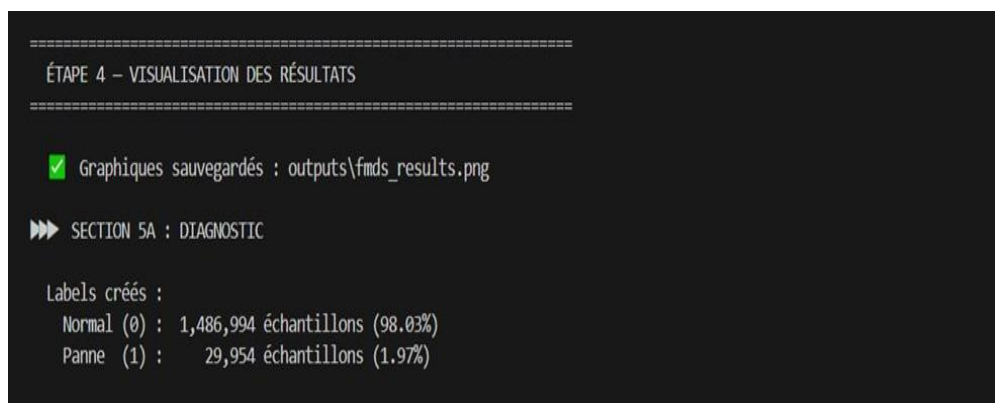


Figure 9: Répartition des états normal / panne

La figure ci-dessous illustre le déséquilibre de classes dans le dataset original, un défi classique en maintenance prédictive industrielle.

Le graphique révèle un déséquilibre très prononcé : les instants de panne représentent seulement 1,97 % du dataset total. Sans rééquilibrage, tout classifieur apprendrait à prédire “normal” en permanence et obtiendrait malgré tout une accuracy de 98 %, ce qui serait trompeur. C’est pourquoi un sous-échantillonnage a été appliqué pour constituer un dataset équilibré de 59 694 échantillons.

6.1.2 Comportement des capteurs

L'analyse temporelle des signaux capteurs révèle des modifications caractéristiques lors des événements de panne. Le tableau suivant synthétise le comportement observé pour chacun des principaux capteurs analogiques en état de fonctionnement normal et lors des périodes de panne (fuites d'air documentées).

Table 10— Comportement des capteurs en état normal et en panne

Capteur	Comportement normal	Comportement en panne
TP2	Oscillations régulières 0–10 bar	Cycles moins réguliers — perte de compression
TP3	Pression stable ≈ 8,9 bar	Chute nette lors de la panne n°3
Reservoirs	Pression proche de TP3 ≈ 9 bar	Chutes plus fréquentes et profondes
DV_pressure	0 en charge, positif en décharge	Anomalies fréquence/amplitude des cycles
Motor_current	Alternance régulière 0 / 4 / 7–9 A	Pics à fort courant plus fréquents

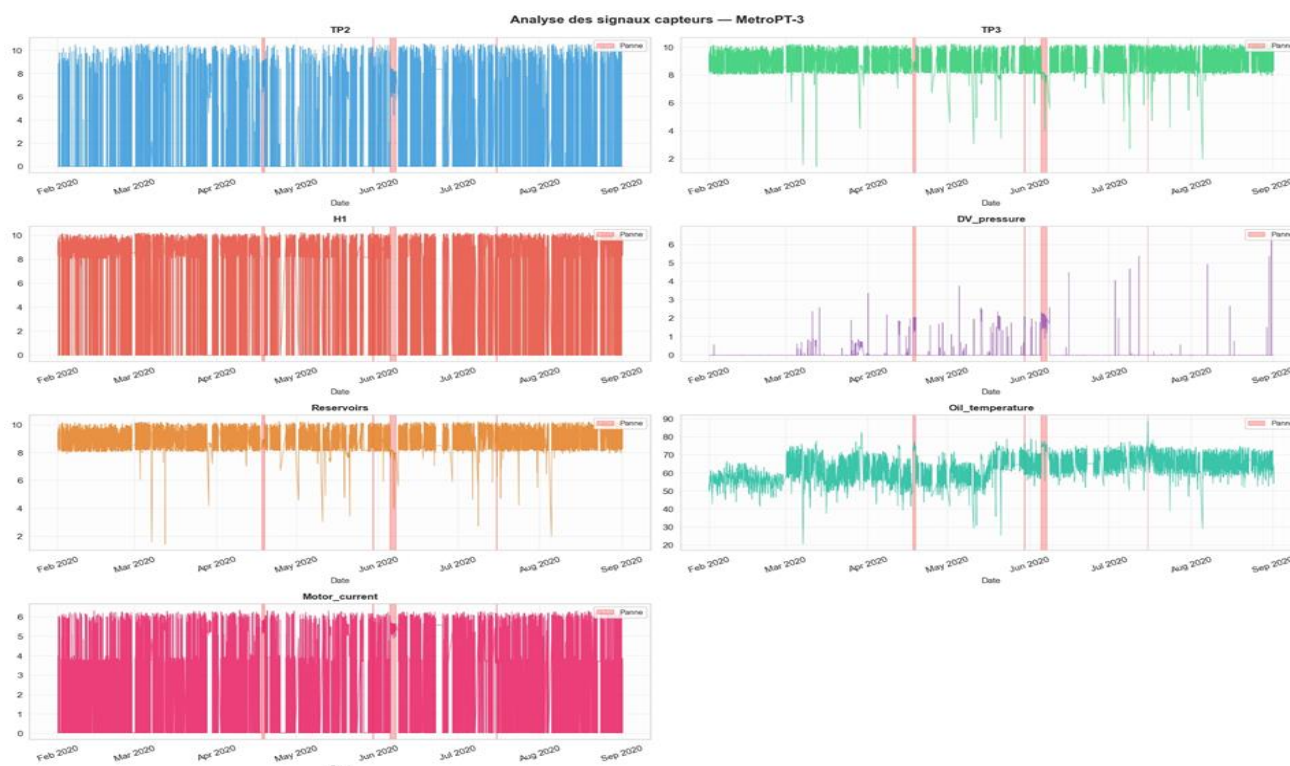


Figure 10: Évolution temporelle des 7 capteurs analogiques avec zones de pannes identifiées

La figure suivante présente l'évolution temporelle des 7 capteurs analogiques sur toute la période d'enregistrement, avec les zones de pannes documentées identifiées en surbrillance.

Les courbes montrent clairement des comportements anormaux lors des pannes : chutes de pression sur TP3 et Reservoirs, cycles irréguliers sur TP2 et DV_pressure, pics de courant moteur. La panne n°3 est la plus visible sur l'ensemble des capteurs, confirmant sa sévérité. Ces observations visuelles valident la pertinence des variables sélectionnées pour l'entraînement des modèles.

6.1.3 Modèles de classification

DIAGNOSTIC – Entraînement des modèles

► SVM (RBF) ...
Accuracy : 0.9925 | F1-score : 0.9925

	precision	recall	f1-score	support
Normal	0.99	0.99	0.99	5948
Panne	0.99	0.99	0.99	5991
accuracy			0.99	11939
macro avg	0.99	0.99	0.99	11939
weighted avg	0.99	0.99	0.99	11939

Résultats de classification — SVM (RBF)

► Arbre de décision ...
Accuracy : 0.9939 | F1-score : 0.9939

	precision	recall	f1-score	support
Normal	0.99	0.99	0.99	5948
Panne	0.99	0.99	0.99	5991
accuracy			0.99	11939
macro avg	0.99	0.99	0.99	11939
weighted avg	0.99	0.99	0.99	11939

► Random Forest ...
Accuracy : 0.9945 | F1-score : 0.9945
Accuracy : 0.9945 | F1-score : 0.9945

	precision	recall	f1-score	support
Normal	1.00	0.99	0.99	5948
Panne	0.99	1.00	0.99	5991
accuracy			0.99	11939
macro avg	0.99	0.99	0.99	11939
weighted avg	0.99	0.99	0.99	11939

Graphique sauvegardé : outputs\diagnostic_modeles.png

Résultats de classification — Arbre de décision

Figure 11 : Résultats des trois modèles de classification supervisée (SVM, Arbre de décision, Random Forest) sur le dataset équilibré (11 939 instances de test)

Les trois modèles atteignent des performances excellentes (Accuracy > 99 %), confirmant la bonne séparabilité des classes après rééquilibrage. Le Random Forest se distingue avec un Recall de 1,00 pour la classe Panne — soit zéro faux négatif — ce qui en fait le modèle le plus fiable pour la détection des pannes en contexte industriel.

6.2 Pronostic — Estimation du RUL (Remaining Useful Life)

Après la détection de pannes, l'étape suivante consiste à estimer combien de temps le compresseur peut encore fonctionner avant sa prochaine défaillance. Cette durée est le RUL (Remaining Useful Life), calculé selon la formule :

Formule de calcul du RUL

$$\text{RUL}(t) = t_{\text{panne_suivante}} - t_{\text{courant}} \quad (\text{dégradation linéaire entre deux pannes})$$

Paramètres : 1 174 489 lignes avec RUL calculé — RUL min : 0,00 h — RUL max : 1 871,98 h — RUL moyen : 685,03 h.

```
=====
PRONOSTIC – Calcul du RUL (Remaining Useful Life)
=====
Lignes avec RUL calculé : 1,174,489
RUL min   : 0.00 h
RUL max   : 1871.98 h
RUL moyen : 685.03 h

Features pour RUL   : 21
Train : 46,980 | Test : 11,745

=====
PRONOSTIC – Entraînement des modèles RUL
=====

▶ Régression linéaire ...
  RMSE : 374.9025 h
  MAE  : 308.2705 h
  R²   : 0.4384

▶ Random Forest ...
  RMSE : 225.8685 h
  MAE  : 172.6857 h
  R²   : 0.7961
✅ Graphique sauvegardé : outputs\pronostic_rul.png
```

Figure 12: Calcul du RUL et paramètres d'entraînement des modèles de pronostic

La figure suivante illustre le calcul du RUL et les paramètres d'entraînement des modèles de pronostic, basés sur la dégradation linéaire entre deux pannes successives.

Le RUL moyen calculé est de 685 heures, correspondant bien au MTBF estimé lors de l'analyse FMDS (681 h). La plage du RUL s'étend de 0 à 1 872 heures, reflétant la grande variabilité des intervalles entre pannes. Cette dispersion constitue un défi pour les modèles de régression, notamment pour les cas proches de la panne.

6.2.1 Modèles de pronostic

Le tableau ci-dessous compare les deux modèles de régression retenus pour l'estimation du RUL, en résumant leur principe de fonctionnement ainsi que leurs avantages et limites respectifs.

Table 11— Modèles de pronostic RUL : principes et comparaison

Modèle	Principe	Avantages / Limites
Régression Linéaire	Relation linéaire capteurs → RUL	Interprétable mais ne capture pas les non-linéarités.
Random Forest ★	Ensemble d'arbres capturant des relations complexes	Plus précis et robuste. Recommandé pour le pronostic.

Le choix de ces deux modèles est justifié par leur complémentarité : la régression linéaire sert de référence (baseline) simple et interprétable, tandis que le Random Forest capture les relations non linéaires entre les capteurs et le RUL. Cette comparaison permet de mesurer concrètement le gain apporté par un modèle plus complexe, et de valider que la sophistication supplémentaire du Random Forest se traduit bien par une amélioration significative des performances de pronostic.

6.2.2 Résultats du pronostic RUL

Le pronostic consiste à estimer le RUL (Remaining Useful Life), c'est-à-dire le temps restant avant la prochaine défaillance. Deux modèles de régression ont été comparés : une régression linéaire (baseline) et un Random Forest. Les paramètres du dataset de pronostic sont : 1 174 489 lignes avec RUL calculé, RUL min = 0,00 h, RUL max = 1 871,98 h, RUL moyen = 685,03 h.

Table 12— Résultats du pronostic RUL (Régression Linéaire vs Random Forest)

Modèle	RMSE (h)	R ²	Interprétation
Régression Linéaire	374,90 h	0,4384	Insuffisant — trop simple
Random Forest ★	225,87 h	0,7961	Explique 79,6 % de la variance du RUL. Recommandé.

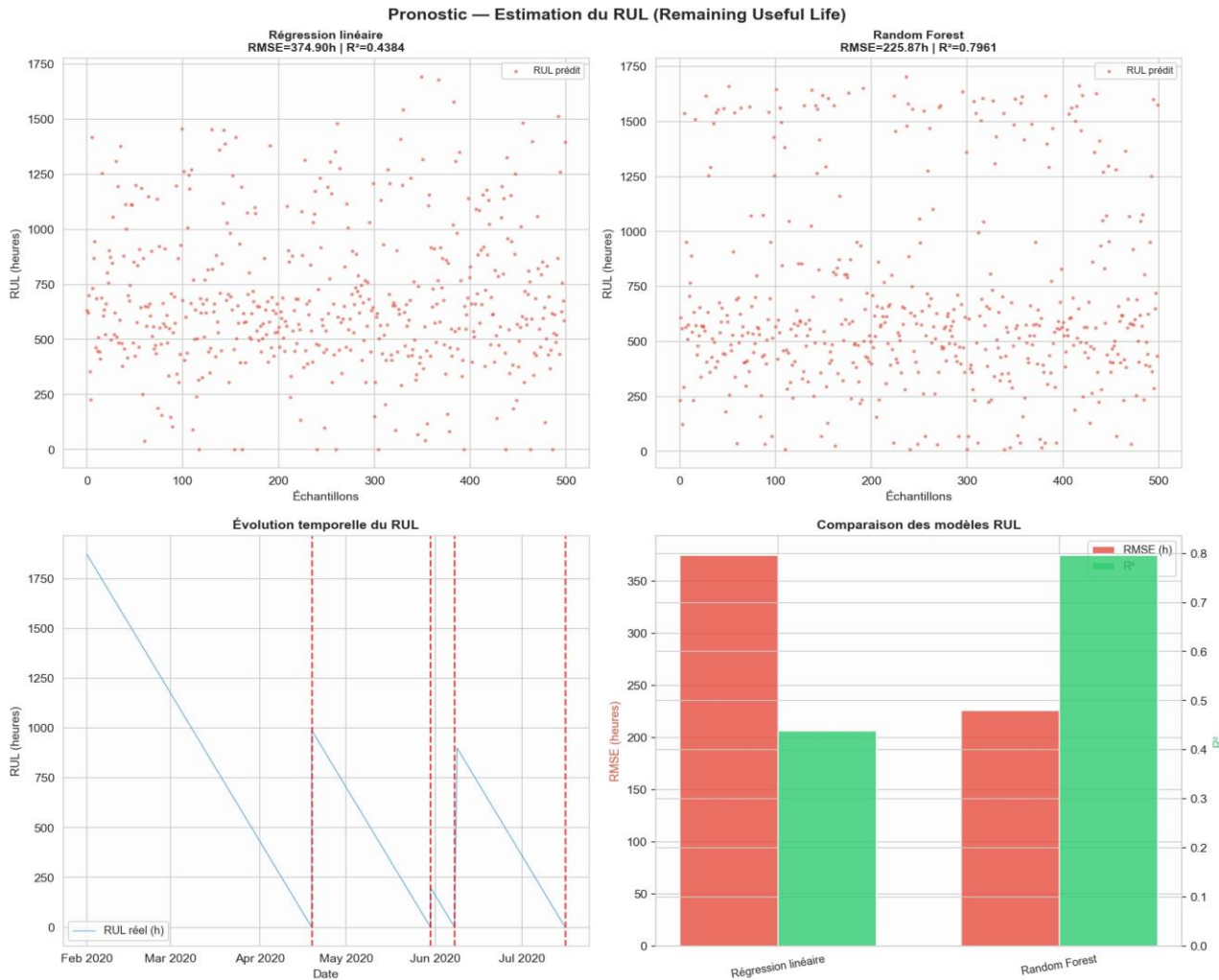


Figure 13: Pronostic RUL : prédictions des modèles, évolution temporelle et comparaison des performances

La figure ci-après compare les prédictions des deux modèles de pronostic (Régression Linéaire et Random Forest) avec les valeurs réelles du RUL sur l'ensemble de test.

Le Random Forest prédit le RUL avec un R² de 0,796 et un RMSE de 225,87 heures, soit une nette supériorité sur la régression linéaire (R² = 0,44). Le RUL moyen calculé de 685 heures est cohérent avec le MTBF FMDS de 681 heures, ce qui valide la cohérence des résultats. Malgré cette

amélioration, une erreur résiduelle de ± 225 heures reste significative, ce qui suggère qu'un modèle plus complexe (LSTM, réseau de neurones temporel) pourrait encore améliorer la précision du pronostic.

Conclusion Pronostic : Le Random Forest est retenu pour l'estimation du RUL ($R^2 = 0,796$, RMSE = 225,87 h). Il permet de planifier les interventions avec une précision exploitable industriellement.

7. Validation des Modèles

7.1 Métriques de Classification

Trois modèles supervisés ont été entraînés sur le dataset équilibré (59 694 échantillons) et évalués sur l'ensemble de test (20 %, soit 11 939 instances). Le tableau ci-dessous compare leurs performances sur les quatre métriques standard de classification binaire.

Table 13— Métriques de classification des trois modèles

Modèle	Accuracy	Précision	Rappel	F1-score
SVM (RBF)	0,9925	0,9925	0,9925	0,9925
Arbre de décision	0,9939	0,9933	0,9945	0,9939
Random Forest ★	0,9945	0,9935	0,9955	0,9945

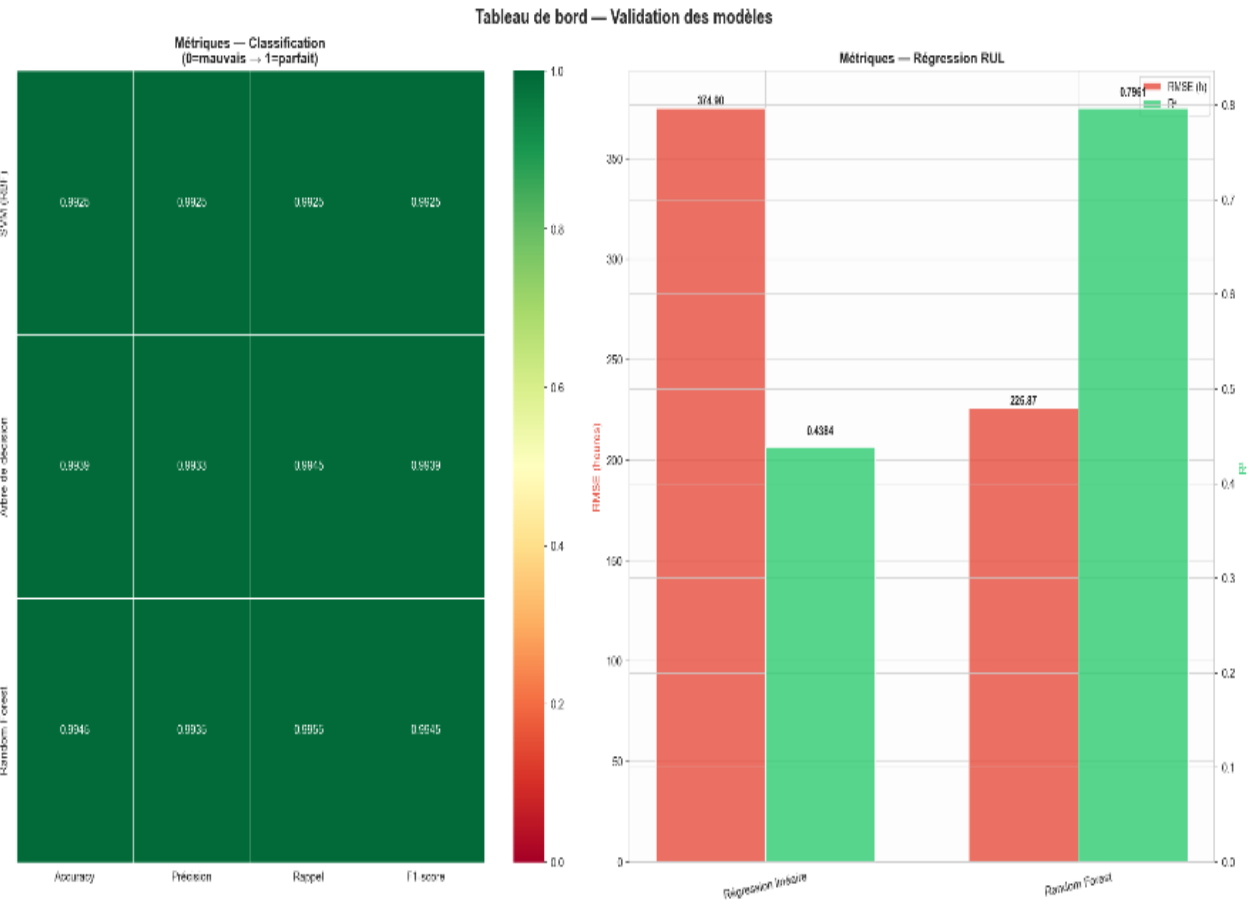


Figure 14: Tableau de bord de validation : métriques de classification (heatmap) et régression RUL

Le figure suivant regroupe l'ensemble des métriques de validation pour les modèles de classification et de régression RUL.

La heatmap confirme l'homogénéité des performances des trois classifieurs, tous supérieurs à 99 % sur les quatre métriques. Le Random Forest domine sur chaque critère et est cohérent entre Precision et Recall, ce qui indique un bon équilibre entre faux positifs et faux négatifs — une propriété cruciale dans un contexte industriel où les deux types d'erreur ont des conséquences opérationnelles.

7.2 Matrices de Confusion

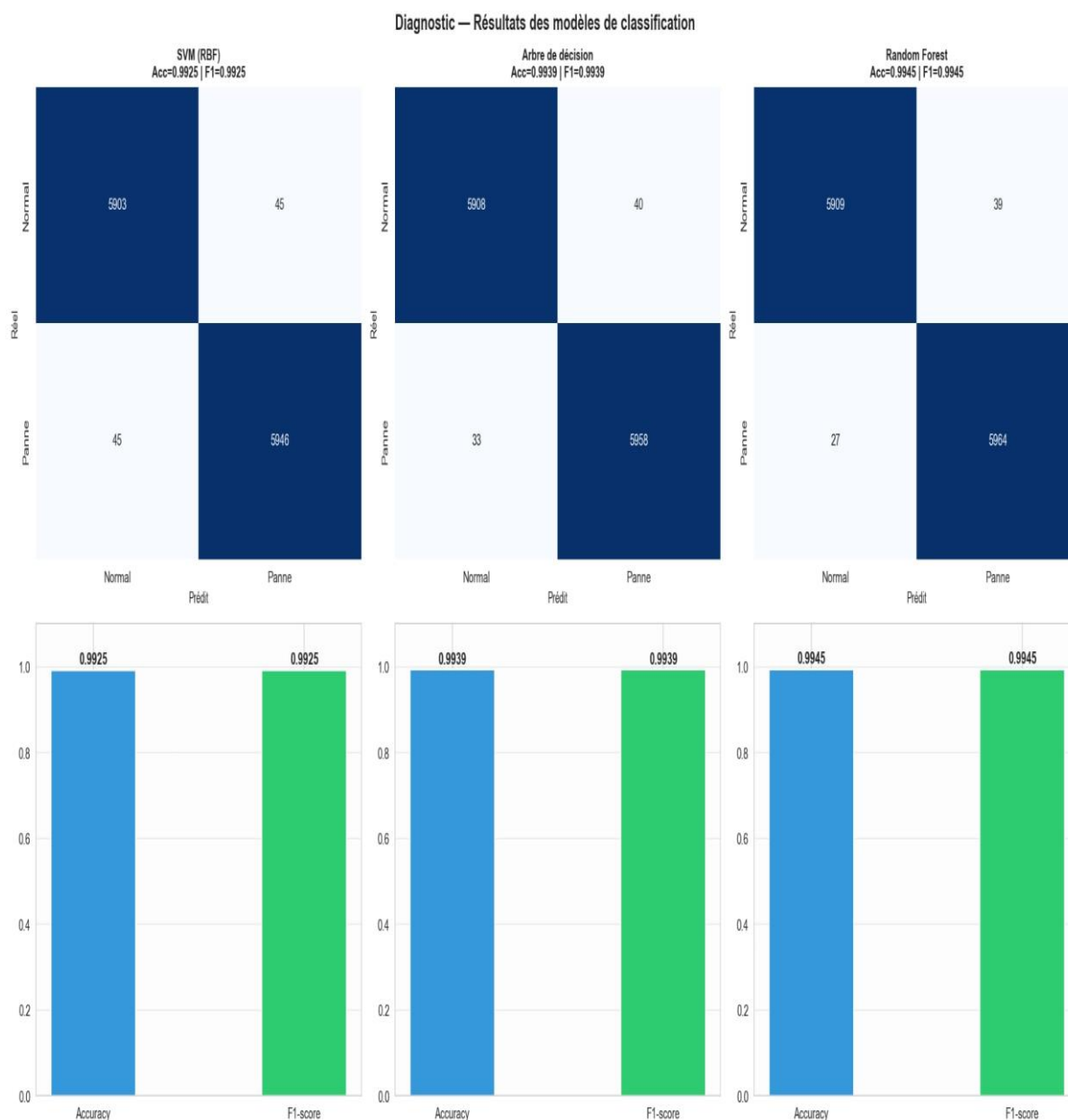


Figure 15 : Matrices de confusion et scores des trois modèles de classification

Les matrices de confusion ci-dessous détaillent la répartition des prédictions correctes et incorrectes pour chacun des trois classifieurs sur les 11 939 instances de test.

Les matrices montrent que le Random Forest ne commet que 66 erreurs au total (39 faux négatifs + 27 faux positifs) sur 11 939 tests, ce qui représente un taux d'erreur inférieur à 0,6 %. Les faux

négatifs (pannes non détectées) sont plus pénalisants que les faux positifs (fausses alertes) dans le contexte ferroviaire, et le Random Forest présente également le rappel le plus élevé (99,55 %), ce qui minimise les pannes manquées.

Le Random Forest ne commet que 66 erreurs sur 11 939 instants testés (39 faux négatifs + 27 faux positifs). C'est le modèle le plus performant pour la détection des pannes.

7.3 Courbes ROC

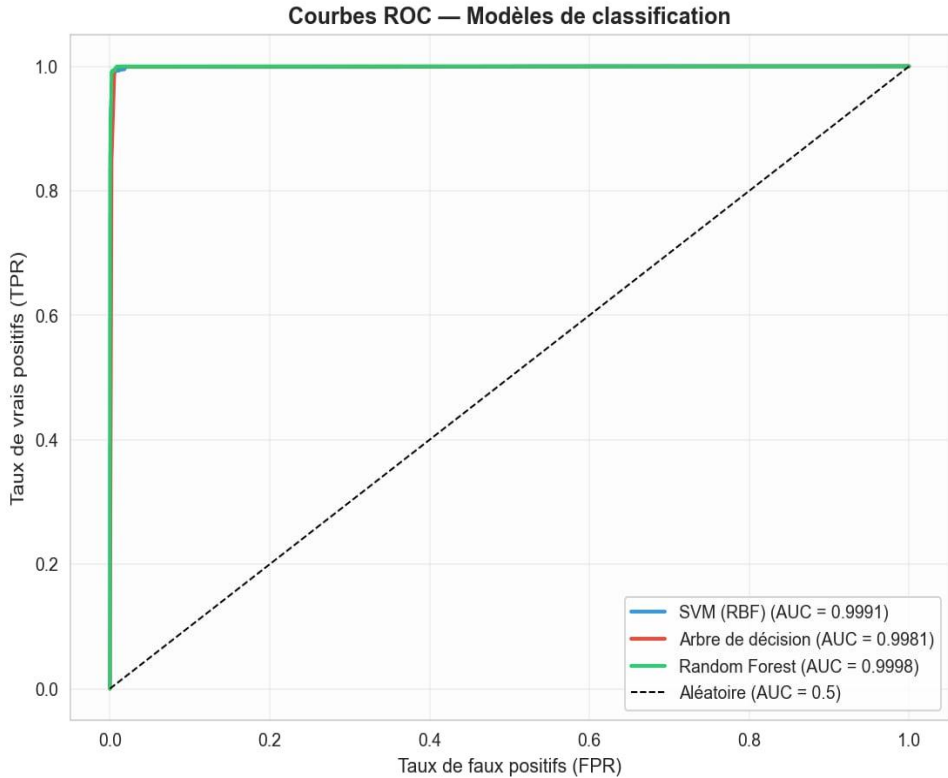


Figure 16: Courbes ROC des trois modèles de classification

La figure suivante présente les courbes ROC (Receiver Operating Characteristic) pour les trois modèles, permettant d'évaluer leur capacité à discriminer les deux classes à différents seuils de décision.

Les trois courbes ROC sont quasi-parfaites, avec des AUC supérieures à 0,998. Le Random Forest atteint une AUC de 0,9998, confirmant qu'il sépare presque parfaitement les états normal et panne. Ces résultats renforcent la confiance dans la capacité du modèle à fonctionner à différents seuils d'alarme, selon les compromis acceptés par l'exploitant.

Table 14— Scores AUC-ROC des trois modèles de classification

Modèle	AUC-ROC	Interprétation
SVM (RBF)	0,9991	Excellent
Arbre de décision	0,9981	Excellent
Random Forest ★	0,9998	Quasi-parfait

7.4 Courbes d'Apprentissage

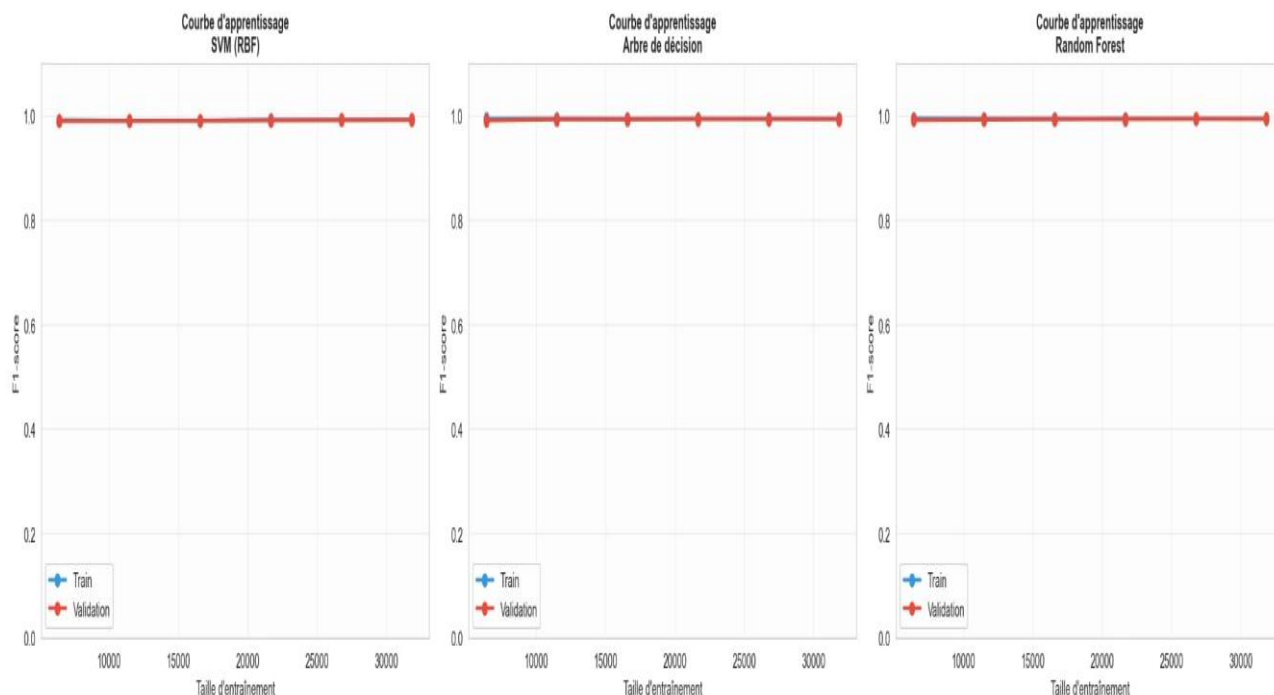


Figure 17: Courbes d'apprentissage des trois modèles (SVM, Arbre de décision, Random Forest)

Les courbes d'apprentissage ci-dessous évaluent la robustesse des modèles en fonction de la taille du jeu d'entraînement.

Les courbes montrent que les trois modèles convergent rapidement, atteignant leur performance maximale avec peu d'échantillons. L'absence de surapprentissage (les courbes de validation rejoignent celles d'entraînement) confirme que les modèles sont fiables et généralisables.

Les trois modèles présentent des courbes stables et proches de 1,0 dès les premières tailles d'entraînement, confirmant leur robustesse et leur bonne généralisation. Aucun signe de surapprentissage.

7.5 Importance des Variables — Random Forest

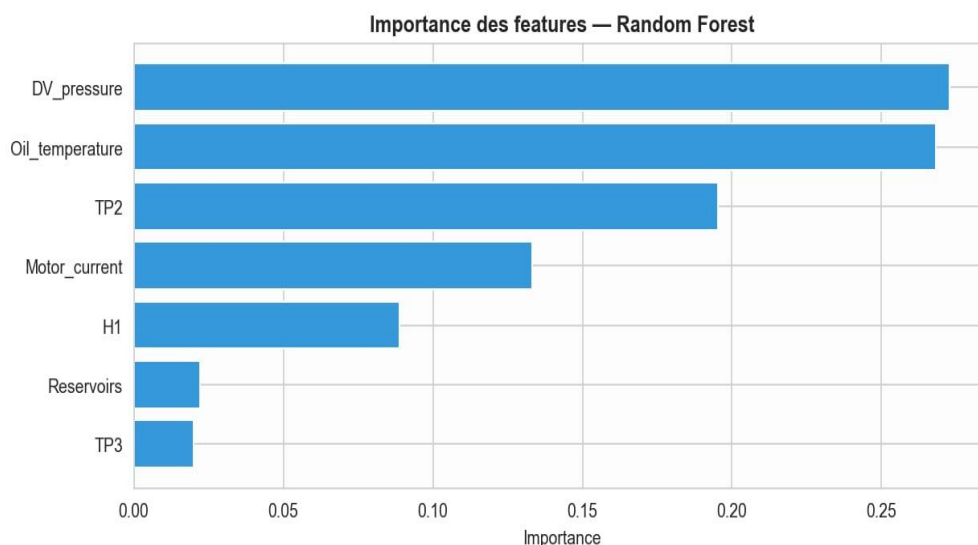


Figure 18: Importance des features — Random Forest

La figure ci-dessous illustre l'importance relative de chaque variable selon le modèle Random Forest, révélant les capteurs les plus déterminants pour la détection des pannes.

DV_pressure (27,27 %) et Oil_temperature (26,81 %) concentrent plus de 54 % de l'importance totale, confirmant leur rôle central dans la détection des fuites d'air. Ce résultat est cohérent avec le comportement physique du système : une fuite provoque une variation de pression dans les sécheurs et un échauffement de l'huile. Ces deux capteurs devraient faire l'objet d'une surveillance renforcée en conditions opérationnelles.

Table 15— Importance des variables selon le modèle Random Forest

Variable	Importance	Signification industrielle
DV_pressure	27,27 %	Capteur le plus discriminant — circuit de séchage
Oil_temperature	26,81 %	Signal thermique précurseur — circuit de lubrification
TP2	19,52 %	Cycles anormaux lors des fuites d'air
Motor_current	13,32 %	Régimes perturbés en cas de panne
H1	8,89 %	Signal secondaire — séparateur cyclonique
Reservoirs + TP3	4,20 %	Contribution individuelle faible

8. Synthèse, Recommandations et Perspectives

8.1 Synthèse Générale des Résultats

Le tableau suivant récapitule les résultats clés obtenus à chaque étape du projet, depuis l'analyse FMDS jusqu'aux modèles de diagnostic et de pronostic, offrant une vue synthétique des performances globales du système de maintenance prédictive développé.

Table 16— Synthèse générale des résultats du projet

Étape	Résultat clé	Conclusion
Analyse FMDS	A = 96,89 % MTBF = 681 h MTTR = 21,87 h	Bonnes performances globales. Le MTTR reste le principal point d'amélioration.
Diagnostic (classification)	Random Forest Acc = 99,45 % AUC = 0,9998	Détection quasi-parfaite des pannes avec seulement 27 erreurs sur 11 939 tests.
Pronostic (RUL)	Random Forest RMSE = 225,87 h R ² = 0,796	Estimation correcte du RUL. Améliorable avec plus de données de panne.
Modélisation Weibull	$\beta = 1,714$ $\eta = 757,5$ h MTBF = 675,5 h	Valide les indicateurs FMDS avec écart < 1 %. Usure progressive confirmée.



Figure 19: Synthèse générale du projet : indicateurs FMDS, performances des modèles, limites et perspectives

La figure suivante présente une vue d'ensemble des résultats obtenus à chaque étape du projet, des indicateurs FMDS jusqu'aux performances des modèles.

Cette synthèse confirme la cohérence globale du projet : les indicateurs FMDS établissent un diagnostic fiable du système, le Random Forest assure une détection quasi-parfaite des pannes, et le pronostic RUL fournit une estimation exploitable de la durée de vie résiduelle. L'ensemble constitue une base solide pour implémenter une maintenance prédictive opérationnelle sur ce type d'équipement ferroviaire.

8.2 Recommandations Pratiques

- **Déployer le modèle Random Forest** : Avec Accuracy = 99,45 % et AUC = 0,9998, il est prêt pour un système de surveillance en temps réel.
- **Surveiller en priorité DV_pressure et Oil_temperature** : Ces deux capteurs concentrent > 54 % de l'importance. Tout comportement anormal doit déclencher une alerte.
- **Planifier les interventions avant 28 jours** : Le MTBF de 681 h (28,4 j) suggère une vérification préventive tous les 20–25 jours.
- **Améliorer la documentation des pannes** : Enregistrer précisément les instants et la nature des pannes pour améliorer la qualité des labels futurs.
- **Planifier les interventions avant $t = 600$ h** : la courbe $R(t)$ Weibull montre que la fiabilité tombe en dessous de 50 % au-delà de cette durée.

8.3 Limites des Approches

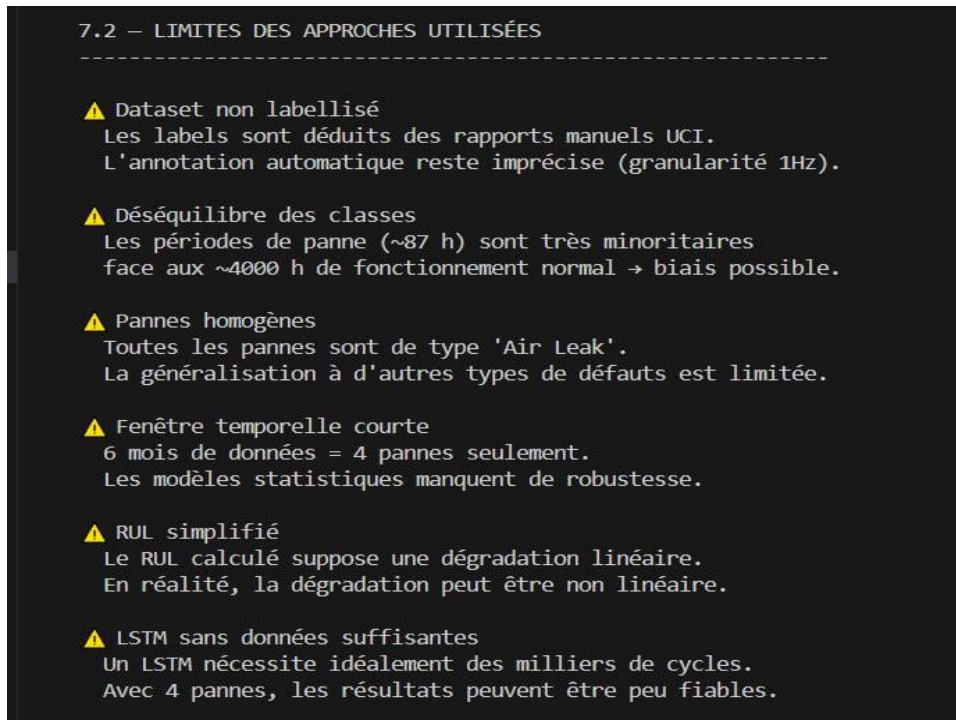


Figure 20: Limites des approches utilisées

La figure ci-dessous identifie les principales limites des approches utilisées dans ce projet.

Les limites identifiées soulignent que les modèles actuels sont entraînés sur un seul type de panne (fuite d'air) avec seulement 4 événements documentés, ce qui restreint leur généralisation. Le modèle de pronostic RUL, bien que fonctionnel, bénéficierait d'une base de données plus riche en événements de dégradation pour affiner ses estimations, Weibull sur échantillon limité — La modélisation repose sur 3 intervalles TBF uniquement, ce qui réduit la robustesse statistique de l'ajustement.

8.4 Perspectives d'Amélioration

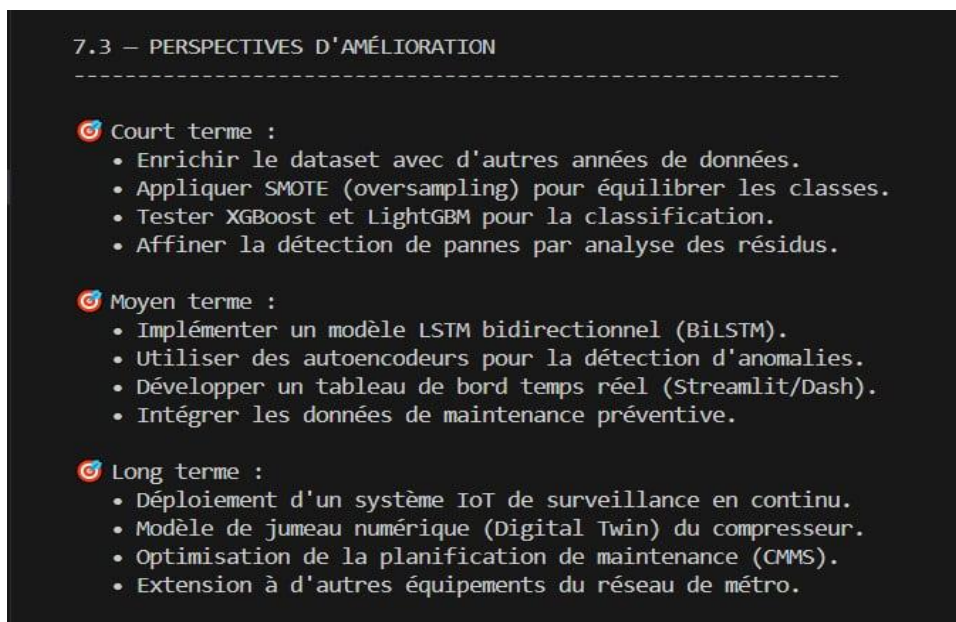


Figure 21: Perspectives d'amélioration à court, moyen et long terme

La figure suivante présente les pistes d'amélioration envisagées à court, moyen et long terme pour renforcer le système de maintenance prédictive.

Les perspectives proposées suivent une progression logique : à court terme, déploiement du Random Forest en temps réel ; à moyen terme, intégration de modèles séquentiels (LSTM) capturant la dynamique temporelle - à long terme, extension à d'autres types de défaillances et généralisation à d'autres rames. Cette roadmap permet de transformer la maintenance corrective actuelle en une approche prédictive optimisée.

Conclusion générale

Ce projet démontre la faisabilité d'une maintenance prédictive performante pour le compresseur d'air du métro de Porto. Le Random Forest se distingue pour la détection des pannes avec un F1-score de 99,45 % et une AUC-ROC de 0,9998, et pour le pronostic avec un R^2 de 0,796 et un RMSE de 225,87 heures. L'analyse FMDS révèle une disponibilité de 96,89 %, un MTBF de 681,17 heures et un MTTR de 21,87 heures. La modélisation Weibull ($\beta = 1,714$ | $\eta = 757,5$ h) confirme une usure progressive du système et valide les indicateurs FMDS avec un écart inférieur à 1 %. Une intervention préventive est recommandée avant $t = 600$ h pour maintenir un niveau de fiabilité acceptable. L'ensemble de ces résultats constitue une base solide pour transformer la maintenance corrective actuelle en une approche prédictive planifiée et économiquement optimisée.

9. Bibliographie et Annexes

9.1 Références Bibliographiques

- [1] Davari, N., Veloso, B., Ribeiro, R., & Gama, J. (2023). MetroPT-3 Dataset. UCI Machine Learning Repository. <https://doi.org/10.24432/C5VW3R>
- [2] Davari, N. et al. (2021). Predictive maintenance based on anomaly detection using deep learning for air production unit in the railway industry. IEEE DSAA 2021.
- [3] Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5–32.
- [4] Cortes, C., & Vapnik, V. (1995). Support-vector networks. Machine Learning, 20(3), 273–297.
- [5] Saxena, A., & Goebel, K. (2008). Damage Propagation Modeling for Aircraft Engine Run-to-Failure Simulation. IEEE PHM.
- [6] ISO 13381-1:2004. Condition monitoring and diagnostics of machines — Prognostics — General guidelines.

9.2 Annexes A : Code de réalisation

A.1 main.py → Point d'entrée du projet

```

=====
main.py — Point d'entrée principal (Version complète)
Projet : Maintenance Prédictive — MetroPT-3
=====

import os, sys
import pandas as pd

from fmds_analysis import load_data, build_failures_table, calculate_fmds, plot_fmds,
section4b_weibull
from diagnostic import create_labels, prepare_data, train_models, analyse_signaux,
plot_diagnostic
from pronostic import compute_rul, prepare_rul_data, train_rul_models, train_lstm_rul,
plot_pronostic
from validation import evaluate_classification, evaluate_regression, plot_roc_curves,
plot_learning_curves, plot_validation_dashboard
from synthese import generer_rapport, plot_synthese

CSV_PATH = os.path.join('data', 'MetroPT3(AirCompressor).csv')
OUTPUT_DIR = 'outputs'
ACTIVER_LSTM = False # True si tensorflow installé

def check_csv(path):
    if os.path.exists(path) and os.path.getsize(path) > 10_000:
        print(f"✓ Fichier CSV trouvé : {path}")
        return True
    print("⚠ Fichier CSV introuvable.")
    print(" Télécharge-le ici :)")
    print(" https://archive.ics.uci.edu/static/public/791/metropt+3+dataset.zip")
    print(f" Puis place MetroPT3(AirCompressor).csv dans : {os.path.abspath('data/')}")
    return False

if __name__ == '__main__':
    os.makedirs(OUTPUT_DIR, exist_ok=True)
    os.makedirs('data', exist_ok=True)

```

```

print("\n ┌───────────────────────────────────────────────────────────────────────────────────┐")
)
print(" │ PROJET MAINTENANCE PRÉDICTIVE — MetroPT-3 │ ")

print(" └───────────────────────────────────────────────────────────────────────────────────┘")

if not check_csv(CSV_PATH):
    sys.exit(1)

# — SECTIONS 1-4 : FMDS —————
print("\n▶▶▶ SECTIONS 1-4 : ANALYSE FMDS")
df = load_data(CSV_PATH)
failures = build_failures_table()
failures = failures[
    (failures['debut'] >= df['timestamp'].min()) &
    (failures['fin'] <= df['timestamp'].max())
].reset_index(drop=True)
resultats_fmfs, TBF_list = calculate_fmfs(df, failures)
plot_fmfs(df, failures, resultats_fmfs, TBF_list, output_dir=OUTPUT_DIR)

# — SECTION 4B : WEIBULL —————
print("\n▶▶▶ SECTION 4B : LOI DE WEIBULL")
weibull_res = section4b_weibull(TBF_list)

# — SECTION 5A : DIAGNOSTIC —————
print("\n▶▶▶ SECTION 5A : DIAGNOSTIC")
df_label = create_labels(df)
analyse_signaux(df_label, output_dir=OUTPUT_DIR)
X_train_c, X_test_c, y_train_c, y_test_c, scaler_c, feat_cols = prepare_data(df_label,
sample_rate=50)
resultats_classif = train_models(X_train_c, X_test_c, y_train_c, y_test_c)
for nom in resultats_classif:
    resultats_classif[nom]['y_test'] = y_test_c.values
plot_diagnostic(resultats_classif, y_test_c, output_dir=OUTPUT_DIR)

# — SECTION 5B : PRONOSTIC —————
print("\n▶▶▶ SECTION 5B : PRONOSTIC RUL")
df_rul = compute_rul(df)
X_train_r, X_test_r, y_train_r, y_test_r, scaler_r = prepare_rul_data(df_rul, sample_rate=20)
resultats_reg = train_rul_models(X_train_r, X_test_r, y_train_r, y_test_r)
for nom in resultats_reg:
    resultats_reg[nom]['y_true'] = y_test_r.values
lstm_model, lstm_history, lstm_res = None, None, None
if ACTIVER_LSTM:
    lstm_model, lstm_history, lstm_res = train_lstm_rul(df_rul, output_dir=OUTPUT_DIR)
plot_pronostic(df_rul, resultats_reg, lstm_res, output_dir=OUTPUT_DIR)

# — SECTION 6 : VALIDATION —————
print("\n▶▶▶ SECTION 6 : VALIDATION DES MODÈLES")
df_metriques_c = evaluate_classification(resultats_classif, X_test_c, y_test_c,
output_dir=OUTPUT_DIR)
df_metriques_r = evaluate_regression(resultats_reg, lstm_res, output_dir=OUTPUT_DIR)
plot_roc_curves(resultats_classif, X_test_c, y_test_c, output_dir=OUTPUT_DIR)
plot_learning_curves(resultats_classif, X_train_c, y_train_c, output_dir=OUTPUT_DIR)

```

```

plot_validation_dashboard(df_metriques_c, df_metriques_r, output_dir=OUTPUT_DIR)

# — SECTION 7 : SYNTHÈSE —
print("\n▶▶▶ SECTION 7 : SYNTHÈSE ET RECOMMANDATIONS")
generer_rapport(resultats_fmfs, df_metriques_c, df_metriques_r, output_dir=OUTPUT_DIR)
plot_synthese(resultats_fmfs, df_metriques_c, df_metriques_r, output_dir=OUTPUT_DIR)

print("\n" + "=====")
)
print("|| PROJET TERMINÉ ✔ — Fichiers dans outputs/ ||")

print("||" + "=====||")
for f in ["fmfs_results.png", "section4b_weibull.png",
        "diagnostic_signaux.png", "diagnostic_modeles.png",
        "diagnostic_feature_importance.png", "pronostic_rul.png",
        "validation_roc.png", "validation_learning_curves.png",
        "validation_dashboard.png", "synthese_finale.png",
        "rapport_synthese.txt"]:
    print(f"|| 📄 {f[:46]} ||")

print("||" + "=====||\n")
)

```

A.2 fmfs_analysis.py → Calcul des indicateurs FMFS

```

"""
=====
fmfs_analysis.py
Module d'analyse FMFS — MetroPT-3 Air Compressor
=====
Contient :
- load_data()      : chargement et exploration du CSV
- build_failures_table(): tableau des 4 pannes officielles
- calculate_fmfs()  : calcul MTBF, MTTR, λ, μ, Disponibilité
- plot_fmfs()      : 6 graphiques de visualisation
=====
"""

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
import matplotlib.patches as mpatches
import seaborn as sns
import os
import math
from scipy.stats import weibull_min

# =====
# PANNES DOCUMENTÉES — rapport officiel UCI MetroPT-3
# =====
# Source : https://archive.ics.uci.edu/dataset/791/metropt+3+dataset
# Le dataset est NON LABELLISÉ, mais les rapports de pannes
# sont fournis par la compagnie de métro de Porto (Portugal).

FAILURE_REPORTS = [

```

```
{
  'id'    : 1,
  'debut' : '2020-04-18 00:00:00',
  'fin'   : '2020-04-18 23:59:00',
  'type'  : 'Air Leak',
  'severite': 'High stress',
  'remarque': 'Panne #1'
},
{
  'id'    : 2,
  'debut' : '2020-05-29 23:30:00',
  'fin'   : '2020-05-30 06:00:00',
  'type'  : 'Air Leak',
  'severite': 'High stress',
  'remarque': 'Maintenance le 30 Avr à 12:00'
},
{
  'id'    : 3,
  'debut' : '2020-06-05 10:00:00',
  'fin'   : '2020-06-07 14:30:00',
  'type'  : 'Air Leak',
  'severite': 'High stress',
  'remarque': 'Maintenance le 8 Jun à 16:00'
},
{
  'id'    : 4,
  'debut' : '2020-07-15 14:30:00',
  'fin'   : '2020-07-15 19:00:00',
  'type'  : 'Air Leak',
  'severite': 'High stress',
  'remarque': 'Maintenance le 16 Jul à 00:00'
}
]
```

```
# =====
# 1. CHARGEMENT ET EXPLORATION DES DONNÉES
# =====
```

```
def load_data(filepath):
```

```
    """
```

```
    Charge le dataset MetroPT-3 depuis le fichier CSV.
```

```
    Colonnes attendues :
```

```
    index, timestamp, TP2, TP3, H1, DV_pressure, Reservoirs,
    Oil_temperature, Motor_current, COMP, DV_eletric, TOWERS,
    MPG, LPS, Pressure_switch, Oil_level, Caudal_impulse
```

```
    Paramètres
```

```
    -----
```

```
    filepath : str
```

```
        Chemin vers le fichier CSV
```

```
    Retourne
```

```
    -----
```

```
    df : pd.DataFrame
```

```

Dataset trié par timestamp
"""
print("=" * 65)
print(" ÉTAPE 1 — CHARGEMENT DU DATASET MetroPT-3")
print("=" * 65)

# Lecture avec gestion automatique de l'encodage
try:
    df = pd.read_csv(filepath, parse_dates=['timestamp'])
except UnicodeDecodeError:
    df = pd.read_csv(filepath, parse_dates=['timestamp'], encoding='latin-1')
except Exception as e:
    raise RuntimeError(f"Erreur de lecture du fichier : {e}")

# Supprimer la colonne index automatique si présente
if 'Unnamed: 0' in df.columns:
    df = df.drop(columns=['Unnamed: 0'])

# Trier par timestamp
df = df.sort_values('timestamp').reset_index(drop=True)

# — Affichage des informations générales —
duree_jours = (df['timestamp'].max() - df['timestamp'].min()).days
print(f"\n ✓ Fichier chargé avec succès")
print(f" {Fichier':<25}: {filepath}")
print(f" {Lignes × Colonnes':<25}: {df.shape[0]:,} × {df.shape[1]:,}")
print(f" {Début':<25}: {df['timestamp'].min():,}")
print(f" {Fin':<25}: {df['timestamp'].max():,}")
print(f" {Durée totale':<25}: {duree_jours} jours")

print(f"\n Colonnes disponibles :\n {list(df.columns)}")

print(f"\n Types de données :")
for col, dtype in df.dtypes.items():
    print(f" {col:<25}: {dtype}")

print(f"\n Valeurs manquantes :")
manquantes = df.isnull().sum()
if manquantes.sum() == 0:
    print(f" Aucune valeur manquante ✓")
else:
    print(manquantes[manquantes > 0])

print(f"\n Statistiques descriptives :")
print(df.describe().round(3).to_string())

return df

# =====
# 2. CONSTRUCTION DU TABLEAU DES PANNES
# =====

def build_failures_table():
    """
    Construit le tableau des pannes à partir des rapports officiels UCI.

```

Calcule la durée TTR (Time To Repair) de chaque panne.

Retourne

failures : pd.DataFrame

Tableau avec colonnes : id, debut, fin, type, severite,
TTR_h (durée en heures), remarque

"""

print("\n" + "=" * 65)

print(" ÉTAPE 2 — PANNES DOCUMENTÉES (rapport officiel UCI)")

print("=" * 65)

failures = pd.DataFrame(FAILURE_REPORTS)

failures['debut'] = pd.to_datetime(failures['debut'])

failures['fin'] = pd.to_datetime(failures['fin'])

Durée de chaque réparation en heures (TTR = Time To Repair)

failures['TTR_h'] = (
failures['fin'] - failures['debut']
)dt.total_seconds() / 3600

print(f"\n Nombre de pannes répertoriées : {len(failures)}")

print(f"\n {'#':<5} {'Début':<22} {'Fin':<22} {'TTR (h)':<10} {'Type':<12} {'Remarque'}")

print(" " + "-" * 90)

for _, r in failures.iterrows():

print(f" {'int(r['id']):<5} {str(r['debut']):<22} {str(r['fin']):<22} "
f"{'TTR_h':<10.2f} {r['type']:<12} {r['remarque']}"")

return failures

```
# =====
# 3. CALCUL DES INDICATEURS FMDS
# =====
```

def calculate_fmfs(df, failures):

"""

Calcule tous les indicateurs FMDS :

- MTBF : Mean Time Between Failures (h)
- MTTR : Mean Time To Repair (h)
- λ : Taux de défaillance (pannes/h)
- μ : Taux de réparation (réparations/h)
- A : Taux de disponibilité (%)

Paramètres

df : pd.DataFrame — dataset complet

failures : pd.DataFrame — tableau des pannes

Retourne

resultats : dict — tous les indicateurs calculés

TBF_list : list — temps entre pannes successives (h)

"""

print("\n" + "=" * 65)

print(" ÉTAPE 3 — CALCUL DES INDICATEURS FMDS")

```

print("=" * 65)

# — Temps total d'observation —
t_debut      = df['timestamp'].min()
t_fin        = df['timestamp'].max()
temps_total_h = (t_fin - t_debut).total_seconds() / 3600

# — Temps total en panne —
temps_panne_h = failures['TTR_h'].sum()

# — Temps total de bon fonctionnement —
temps_fonct_h = temps_total_h - temps_panne_h

# — Nombre de pannes —
nb_pannes     = len(failures)

# — TBF : Temps Between Failures (entre pannes consécutives) —
TBF_list = []
for i in range(1, len(failures)):
    fin_precedente = failures.iloc[i - 1]['fin']
    debut_suivante = failures.iloc[i]['debut']
    tbf = (debut_suivante - fin_precedente).total_seconds() / 3600
    TBF_list.append(round(tbf, 2))

# — MTBF — Mean Time Between Failures —
# Méthode 1 : moyenne des TBF entre pannes consécutives (recommandée)
MTBF_tbf = np.mean(TBF_list) if TBF_list else 0
# Méthode 2 : temps fonctionnement total / nombre de pannes
MTBF_global = temps_fonct_h / nb_pannes if nb_pannes > 0 else 0
MTBF        = MTBF_tbf # méthode TBF utilisée

# — MTTR — Mean Time To Repair —
MTTR = failures['TTR_h'].mean()

# — Taux de défaillance  $\lambda$  (pannes/heure) —
lambda_ = 1 / MTBF if MTBF > 0 else 0

# — Taux de réparation  $\mu$  (réparations/heure) —
mu_ = 1 / MTTR if MTTR > 0 else 0

# — Disponibilité A (%) —
disponibilite_fmfs = (MTBF / (MTBF + MTTR)) * 100 if (MTBF + MTTR) > 0 else 0
disponibilite_directe = (temps_fonct_h / temps_total_h) * 100 if temps_total_h > 0 else 0
indisponibilite      = 100 - disponibilite_fmfs

# — Dictionnaire des résultats —
resultats = {
    'periode'          : f"{t_debut.date()} → {t_fin.date()}",
    'temps_total_h'     : round(temps_total_h, 2),
    'temps_fonctionnement_h' : round(temps_fonct_h, 2),
    'temps_panne_h'     : round(temps_panne_h, 2),
    'nb_pannes'         : nb_pannes,
    'TBF_list'          : TBF_list,
    'TTR_list'          : failures['TTR_h'].round(2).tolist(),
    'MTBF'              : round(MTBF, 2),
    'MTBF_global'       : round(MTBF_global, 2),

```

```

'MTTR'          : round(MTTR, 2),
'lambda'        : round(lambda_, 8),
'mu'            : round(mu_, 6),
'disponibilite_fmfs' : round(disponibilite_fmfs, 4),
'disponibilite_directe' : round(disponibilite_directe, 4),
'indisponibilite'   : round(indisponibilite, 4),
}

# — Affichage formaté —
print(f"\n {Indicateur':<40} {'Valeur':>18} {'Unité'}")
print(" " + "-" * 70)
lignes = [
    ("Période d'observation", resultats['periode'], ""),
    ("Temps total", resultats['temps_total_h'], "heures"),
    ("Temps de fonctionnement", resultats['temps_fonctionnement_h'], "heures"),
    ("Temps total en panne", resultats['temps_panne_h'], "heures"),
    ("Nombre de pannes", resultats['nb_pannes'], "pannes"),
    ("TBF individuels", str(resultats['TBF_list']), "heures"),
    ("TTR individuels", str(resultats['TTR_list']), "heures"),
    ("MTBF (méthode TBF)", resultats['MTBF'], "heures"),
    ("MTBF (méthode globale)", resultats['MTBF_global'], "heures"),
    ("MTTR", resultats['MTTR'], "heures"),
    ("Taux de défaillance  $\lambda$ ", resultats['lambda'], "pannes/h"),
    ("Taux de réparation  $\mu$ ", resultats['mu'], "rép/h"),
    ("Disponibilité (FMFS)", f"{resultats['disponibilite_fmfs']:.4f}%", ""),
    ("Disponibilité (directe)", f"{resultats['disponibilite_directe']:.4f}%", ""),
    ("Indisponibilité", f"{resultats['indisponibilite']:.4f}%", ""),
]
for label, valeur, unite in lignes:
    print(f" {label:<40} {str(valeur):>18} {unite}")

# — Interprétation —
print(f"\n INTERPRÉTATION :")
print(f" {'-'*60}")
print(f" 🛠 MTBF = {MTBF:.2f} h → le compresseur fonctionne en moyenne")
print(f" {MTBF:.1f} heures ({MTBF/24:.1f} jours) entre deux pannes.")

print(f"\n 🛠 MTTR = {MTTR:.2f} h → une réparation dure en moyenne {MTTR:.1f} heures.")

print(f"\n ⚡  $\lambda$  = {lambda_:.8f} pannes/h")
print(f" ≈ {lambda_*24:.5f} pannes/jour")
print(f" ≈ {lambda_*24*30:.4f} pannes/mois")

print(f"\n 🔄  $\mu$  = {mu:.6f} réparations/h")

niveau = (
    "☐ EXCELLENT (≥ 99%)" if disponibilite_fmfs >= 99 else
    "☐ BON (95–99%)" if disponibilite_fmfs >= 95 else
    "☐ ACCEPTABLE (90–95%)" if disponibilite_fmfs >= 90 else
    "● INSUFFISANT (< 90%)"
)

print(f"\n 📊 Disponibilité = {disponibilite_fmfs:.4f}% — {niveau}")
print(f" Le système est opérationnel {disponibilite_fmfs:.4f}% du temps.")
print(f" Indisponibilité = {indisponibilite:.4f}% du temps.")

```

```

return resultats, TBF_list

# =====
# 4. VISUALISATIONS
# =====

def plot_fmlds(df, failures, resultats, TBF_list, output_dir='outputs'):
    """
    Génère 6 graphiques FMDS et les sauvegarde dans output_dir.

    Graphiques produits :
    1. Timeline pression TP2 + zones de pannes
    2. Durée de réparation par panne (TTR vs MTTR)
    3. Temps entre pannes (TBF vs MTBF)
    4. Évolution Motor_current + zones de pannes
    5. Tableau de synthèse des indicateurs
    6. Camembert disponibilité / indisponibilité

    Paramètres
    -----
    df      : pd.DataFrame
    failures : pd.DataFrame
    resultats : dict
    TBF_list : list
    output_dir : str
    """
    print("\n" + "=" * 65)
    print(" ÉTAPE 4 — VISUALISATION DES RÉSULTATS")
    print("=" * 65)

    os.makedirs(output_dir, exist_ok=True)

    # — Palette de couleurs —
    C_PANNE = '#e74c3c'
    C_OK = '#2ecc71'
    C_BLEU = '#3498db'
    C_ORANGE = '#e67e22'
    C_GRISE = '#95a5a6'
    C_FOND = '#f8f9fa'

    sns.set_style("whitegrid")
    fig = plt.figure(figsize=(22, 18))
    fig.patch.set_facecolor(C_FOND)
    fig.suptitle(
        "Analyse FMDS — Compresseur d'Air MetroPT-3 (Porto, Fév–Août 2020)",
        fontsize=17, fontweight='bold', y=0.98, color='#2c3e50'
    )

    # Sous-échantillonnage (1 pt / 100) pour éviter la surcharge graphique
    df_s = df.iloc[::100].copy()

    # =====
    # GRAPHIQUE 1 — Timeline TP2 + zones de pannes
    # =====
    ax1 = fig.add_subplot(3, 2, (1, 2))

```

```
ax1.plot(
    df_s['timestamp'], df_s['TP2'],
    color=C_BLEU, linewidth=0.7, alpha=0.85,
    label='TP2 — Pression compresseur (bar)'
)

premier = True
for _, row in failures.iterrows():
    lbl = 'Zone de panne (Air Leak)' if premier else "
    ax1.axvspan(row['debut'], row['fin'], alpha=0.30, color=C_PANNE, label=lbl)
    ax1.annotate(
        f" Panne #{int(row['id'])}",
        xy=(row['debut'], ax1.get_ylim()[1]),
        xytext=(0, 6), textcoords='offset points',
        fontsize=9, color=C_PANNE, fontweight='bold'
    )
    premier = False

ax1.set_title('Timeline : Pression TP2 avec zones de pannes identifiées', fontsize=12,
fontweight='bold')
ax1.set_xlabel('Date', fontsize=10)
ax1.set_ylabel('Pression (bar)', fontsize=10)
ax1.xaxis.set_major_formatter(mdates.DateFormatter('%b %Y'))
ax1.xaxis.set_major_locator(mdates.MonthLocator())
plt.setp(ax1.xaxis.get_majorticklabels(), rotation=20)
ax1.legend(loc='lower right', fontsize=9)
ax1.set_facecolor('#fdfdfd')

# =====
# GRAPHIQUE 2 — TTR par panne
# =====

ax2 = fig.add_subplot(3, 2, 3)

ttr_vals = failures['TTR_h'].tolist()
mttr_val = resultats['MTTR']
labels_p = [f"Panne #{int(i+1)}" for i in range(len(ttr_vals))]
couleurs = [C_PANNE if t > mttr_val else C_OK for t in ttr_vals]

bars = ax2.bar(
    labels_p, ttr_vals,
    color=couleurs, edgecolor='white', linewidth=1.2, width=0.5
)
for bar, val in zip(bars, ttr_vals):
    ax2.text(
        bar.get_x() + bar.get_width() / 2,
        bar.get_height() + 0.4,
        f"{val:.1f} h",
        ha='center', va='bottom', fontsize=11, fontweight='bold', color='#2c3e50'
    )

ax2.axhline(
    y=mttr_val, color=C_ORANGE, linestyle='--', linewidth=2,
    label=f"MTTR = {mttr_val:.2f} h"
)
ax2.set_title('Durée de réparation par panne (TTR)', fontsize=12, fontweight='bold')
```

```

ax2.set_xlabel('Panne', fontsize=10)
ax2.set_ylabel('Durée (heures)', fontsize=10)
ax2.legend(fontsize=10)
ax2.set_facecolor('#fdfdfd')

patch_sup = mpatches.Patch(color=C_PANNE, label='> MTTR')
patch_inf = mpatches.Patch(color=C_OK, label='≤ MTTR')
ax2.legend(handles=[ax2.get_lines()[0], patch_sup, patch_inf], fontsize=9)

# =====
# GRAPHIQUE 3 — TBF entre pannes
# =====
ax3 = fig.add_subplot(3, 2, 4)

if TBF_list:
    mtbf_val = resultats['MTBF']
    labels_tbf = [f'TBF {i+1}→{i+2}" for i in range(len(TBF_list))]
    bars2 = ax3.bar(
        labels_tbf, TBF_list,
        color=C_BLEU, edgecolor='white', linewidth=1.2, width=0.4
    )
    for bar, val in zip(bars2, TBF_list):
        ax3.text(
            bar.get_x() + bar.get_width() / 2,
            bar.get_height() + 5,
            f'{val:.0f} h\n({val/24:.0f} j)',
            ha='center', va='bottom', fontsize=10, fontweight='bold', color='#2c3e50'
        )
    ax3.axhline(
        y=mtbf_val, color=C_ORANGE, linestyle='--', linewidth=2,
        label=f'MTBF = {mtbf_val:.2f} h'
    )
    ax3.set_title('Temps entre pannes consécutives (TBF)', fontsize=12, fontweight='bold')
    ax3.set_xlabel('Intervalle', fontsize=10)
    ax3.set_ylabel('Durée (heures)', fontsize=10)
    ax3.legend(fontsize=10)
    ax3.set_facecolor('#fdfdfd')

# =====
# GRAPHIQUE 4 — Motor Current + zones pannes
# =====
ax4 = fig.add_subplot(3, 2, 5)

if 'Motor_current' in df_s.columns:
    ax4.plot(
        df_s['timestamp'], df_s['Motor_current'],
        color='#9b59b6', linewidth=0.7, alpha=0.85,
        label='Courant moteur (A)'
    )
    for _, row in failures.iterrows():
        ax4.axvspan(row['debut'], row['fin'], alpha=0.30, color=C_PANNE)

ax4.set_title('Courant moteur + zones de pannes', fontsize=12, fontweight='bold')
ax4.set_xlabel('Date', fontsize=10)
ax4.set_ylabel('Courant (A)', fontsize=10)
ax4.xaxis.set_major_formatter(mdates.DateFormatter('%b %Y'))

```

```

    ax4.xaxis.set_major_locator(mdates.MonthLocator())
    plt.setp(ax4.xaxis.get_majorticklabels(), rotation=20)
    ax4.legend(fontsize=9)
    ax4.set_facecolor('#fdfdfd')
else:
    ax4.text(0.5, 0.5, "Colonne 'Motor_current' non trouvée",
            ha='center', va='center', transform=ax4.transAxes, fontsize=11)
    ax4.axis('off')

# =====
# GRAPHIQUE 5 — Camembert disponibilité
# =====
ax5 = fig.add_subplot(3, 2, 6)

dispo = resultats['disponibilite_fmfs']
indispo = resultats['indisponibilite']
sizes = [dispo, indispo]
labels_pie = [
    f'Disponible\n{dispo:.4f}%',
    f'Indisponible\n{indispo:.4f}%'
]
]
wedges, texts, autotexts = ax5.pie(
    sizes,
    labels=labels_pie,
    colors=[C_OK, C_PANNE],
    autopct='%1.4f%%',
    startangle=90,
    explode=(0.03, 0.10),
    wedgeprops={'edgecolor': 'white', 'linewidth': 2},
    textprops={'fontsize': 10}
)
for at in autotexts:
    at.set_fontsize(9)
    at.set_fontweight('bold')

ax5.set_title(
    f'Disponibilité du système = {dispo:.4f}%',
    fontsize=12, fontweight='bold'
)

# =====
# TABLEAU DE SYNTHÈSE (annotation sur figure)
# =====
synthese = (
    f" SYNTHÈSE FMFS\n"
    f" {'-'*38}\n"
    f" Période      : {resultats['periode']}\n"
    f" Nb pannes    : {resultats['nb_pannes']}\n"
    f" MTBF         : {resultats['MTBF']:.2f} h ({resultats['MTBF']/24:.1f} jours)\n"
    f" MTRR         : {resultats['MTRR']:.2f} h\n"
    f" λ (défaillance): {resultats['lambda']:.2e} p/h\n"
    f" μ (réparation) : {resultats['mu']:.4f} r/h\n"
    f" Disponibilité : {resultats['disponibilite_fmfs']:.4f} %\n"
    f" Indisponibilité: {resultats['indisponibilite']:.4f} %"
)
fig.text(

```

```

    0.01, 0.02, synthese,
    fontsize=9.5, family='monospace',
    verticalalignment='bottom',
    bbox=dict(boxstyle='round,pad=0.6', facecolor='#ecf0f1', alpha=0.9, edgecolor='#bdc3c7')
)

# — Sauvegarde —
plt.tight_layout(rect=[0, 0.12, 1, 0.97])
chemin_png = os.path.join(output_dir, 'fmds_results.png')
plt.savefig(chemin_png, dpi=150, bbox_inches='tight', facecolor=C_FOND)
plt.show()
print(f"\n ✓ Graphiques sauvegardés : {chemin_png}")

# =====
# 4B. MODÉLISATION WEIBULL
# =====

def section4b_weibull(operating_times, output_dir='outputs'):
    """
    Modélisation Weibull de la fiabilité à partir des TBF.

    Paramètres
    -----
    operating_times : list — temps entre pannes (TBF en heures)
    output_dir      : str  — dossier de sauvegarde

    Retourne
    -----
    dict : beta, eta, mtbf_weibull
    """
    print("\n" + "=" * 62)
    print("  SECTION 4B — Loi de Weibull")
    print("=" * 62)

    if len(operating_times) < 2:
        print("  Pas assez de données pour Weibull.")
        return {}

    os.makedirs(output_dir, exist_ok=True)

    # — Fit Weibull —
    shape, loc, scale = weibull_min.fit(operating_times, floc=0)
    beta    = shape
    eta     = scale
    mtbf_w  = eta * math.gamma(1 + 1 / beta)

    print(f"\n  β (forme)    = {beta:.4f}")
    print(f"  η (échelle)  = {eta:.2f} h")
    print(f"  MTBF Weibull = {mtbf_w:.2f} h")

    if beta < 1:
        interpretation = "β < 1 : Mortalité infantile — taux de défaillance décroissant"
    elif beta <= 2:
        interpretation = "β ≈ 1–2 : Pannes aléatoires / usure progressive"
    else:

```

```

interpretation = " $\beta > 2$  : Usure rapide — taux de défaillance croissant"

print(f" → {interpretation}")

# — Courbes —
t = np.linspace(0, 2000, 1000)
R_t = 1 - weibull_min.cdf(t, beta, loc=0, scale=eta)
F_t = weibull_min.cdf(t, beta, loc=0, scale=eta)
pdf_t = weibull_min.pdf(t, beta, loc=0, scale=eta)
h_t = pdf_t / (R_t + 1e-10)

C_FOND = '#f8f9fa'
fig, axes = plt.subplots(1, 3, figsize=(18, 5), facecolor=C_FOND)
fig.suptitle(
    f"Analyse Weibull — Compresseur MetroPT-3  $\beta={\rm{beta:.3f}}$   $\eta={\rm{eta:.1f}}$ h
    MTBF={mtbf_w:.1f}h",
    fontsize=13, fontweight='bold'
)

# — R(t) — Fiabilité —
axes[0].plot(t, R_t, color='#2ecc71', lw=2.5)
axes[0].axvline(mtbw, color='#2980b9', linestyle='--', lw=2,
    label=f'MTBF = {mtbf_w:.0f} h')
axes[0].fill_between(t, R_t, alpha=0.10, color='#2ecc71')
axes[0].set_title('R(t) — Fiabilité', fontsize=12, fontweight='bold')
axes[0].set_xlabel('t (heures)')
axes[0].set_ylabel('R(t)')
axes[0].legend(fontsize=10)
axes[0].grid(alpha=0.3)
axes[0].set_facecolor(C_FOND)
axes[0].set_ylim(0, 1.05)

# — F(t) — Probabilité de défaillance —
axes[1].plot(t, F_t, color='#e74c3c', lw=2.5)
axes[1].axvline(mtbw, color='#2980b9', linestyle='--', lw=2,
    label=f'MTBF = {mtbf_w:.0f} h')
axes[1].fill_between(t, F_t, alpha=0.10, color='#e74c3c')
axes[1].set_title('F(t) — Probabilité de défaillance', fontsize=12, fontweight='bold')
axes[1].set_xlabel('t (heures)')
axes[1].set_ylabel('F(t)')
axes[1].legend(fontsize=10)
axes[1].grid(alpha=0.3)
axes[1].set_facecolor(C_FOND)
axes[1].set_ylim(0, 1.05)

# — h(t) — Taux de défaillance instantané —
h_clean = np.where(t > 10, h_t, np.nan)
axes[2].plot(t, h_clean, color='#e67e22', lw=2.5)
axes[2].set_title('h(t) — Taux de défaillance instantané', fontsize=12, fontweight='bold')
axes[2].set_xlabel('t (heures)')
axes[2].set_ylabel('h(t)')
axes[2].grid(alpha=0.3)
axes[2].set_facecolor(C_FOND)

# Annotation interprétation
fig.text(0.5, 0.01, f" → {interpretation}",

```

```

        ha='center', fontsize=10, style='italic', color='#555',
        bbox=dict(boxstyle='round,pad=0.4', facecolor='#ecf0f1', alpha=0.8))

plt.tight_layout(rect=[0, 0.06, 1, 0.95])
path = os.path.join(output_dir, 'section4b_weibull.png')
plt.savefig(path, dpi=150, bbox_inches='tight', facecolor=C_FOND)
plt.show()
print(f"\n ✓ Graphique Weibull sauvegardé : {path}")

return {
    'beta'      : round(beta, 4),
    'eta'       : round(eta, 2),
    'mtbf_weibull' : round(mtbw, 2),
    'interpretation': interpretation
}

```

A.3 diagnostic.py → Classification et détection des pannes

```

"""
=====
diagnostic.py
Section 5 — Maintenance Prédictive : DIAGNOSTIC
=====
- Détection des défaillances via SVM et arbres de décision
- Analyse de signaux et capteurs
=====
"""

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
import seaborn as sns
import os
import warnings
warnings.filterwarnings('ignore')

from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (
    classification_report, confusion_matrix,
    accuracy_score, f1_score
)

# =====
# FEATURES CAPTEURS UTILISÉES
# =====
FEATURE_COLS = [
    'TP2', 'TP3', 'H1', 'DV_pressure', 'Reservoirs',
    'Oil_temperature', 'Motor_current'
]

# Pannes officielles UCI

```

```

FAILURE_WINDOWS = [
    ('2020-04-18 00:00:00', '2020-04-18 23:59:00'),
    ('2020-05-29 23:30:00', '2020-05-30 06:00:00'),
    ('2020-06-05 10:00:00', '2020-06-07 14:30:00'),
    ('2020-07-15 14:30:00', '2020-07-15 19:00:00'),
]

# =====
# 1. CRÉATION DES LABELS (0 = Normal, 1 = Panne)
# =====

def create_labels(df):
    """
    Crée une colonne 'label' binaire :
    0 = fonctionnement normal
    1 = période de panne (selon rapport UCI)
    """
    df = df.copy()
    df['label'] = 0

    for debut_str, fin_str in FAILURE_WINDOWS:
        debut = pd.to_datetime(debut_str)
        fin = pd.to_datetime(fin_str)
        mask = (df['timestamp'] >= debut) & (df['timestamp'] <= fin)
        df.loc[mask, 'label'] = 1

    n_pannes = df['label'].sum()
    n_normal = (df['label'] == 0).sum()
    print(f"\n Labels créés :")
    print(f"   Normal (0) : {n_normal:>10,} échantillons ({n_normal/len(df)*100:.2f}%)")
    print(f"   Panne (1) : {n_pannes:>10,} échantillons ({n_pannes/len(df)*100:.2f}%)")

    return df

# =====
# 2. PRÉPARATION DES DONNÉES
# =====

def prepare_data(df, sample_rate=50):
    """
    Sous-échantillonne et prépare X, y pour la classification.

    Paramètres
    -----
    df : DataFrame avec colonne 'label'
    sample_rate : garder 1 ligne sur N (défaut 50)
    """
    print("\n" + "=" * 65)
    print(" DIAGNOSTIC — Préparation des données")
    print("=" * 65)

    # Sous-échantillonnage équilibré
    df_panne = df[df['label'] == 1]
    df_normal = df[df['label'] == 0].iloc[::sample_rate]

```

```

df_model = pd.concat([df_normal, df_panne]).sample(frac=1, random_state=42)

# Garder uniquement les features disponibles
cols_dispo = [c for c in FEATURE_COLS if c in df_model.columns]
X = df_model[cols_dispo].fillna(df_model[cols_dispo].median())
y = df_model['label']

print(f" Features utilisées : {cols_dispo}")
print(f" Échantillons totaux : {len(X):,}")
print(f" Ratio panne/normal : {y.sum()}/{(y==0).sum()}")

# Split train / test (80 / 20)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Normalisation
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

print(f" Train : {len(X_train):,} | Test : {len(X_test):,}")

return X_train, X_test, y_train, y_test, scaler, cols_dispo

# =====
# 3. ENTRAÎNEMENT DES MODÈLES
# =====

def train_models(X_train, X_test, y_train, y_test):
    """
    Entraîne SVM, Arbre de décision et Random Forest.
    Retourne les résultats de chaque modèle.
    """
    print("\n" + "=" * 65)
    print(" DIAGNOSTIC — Entraînement des modèles")
    print("=" * 65)

    modeles = {
        'SVM (RBF)' : SVC(kernel='rbf', C=1.0, gamma='scale', random_state=42),
        'Arbre de décision' : DecisionTreeClassifier(max_depth=6, random_state=42),
        'Random Forest' : RandomForestClassifier(n_estimators=100, max_depth=8,
random_state=42, n_jobs=-1),
    }

    resultats = {}
    for nom, modele in modeles.items():
        print(f"\n ► {nom} ...")
        modele.fit(X_train, y_train)
        y_pred = modele.predict(X_test)

        acc = accuracy_score(y_test, y_pred)
        f1 = f1_score(y_test, y_pred, average='weighted')
        cm = confusion_matrix(y_test, y_pred)

```

```

    resultats[nom] = {
        'modele' : modele,
        'y_pred' : y_pred,
        'acc' : acc,
        'f1' : f1,
        'cm' : cm,
        'report' : classification_report(y_test, y_pred,
                                         target_names=['Normal', 'Panne'])
    }

    print(f"    Accuracy : {acc:.4f} | F1-score : {f1:.4f}")
    print(f"\n{resultats[nom]}['report']")

return resultats

# =====
# 4. ANALYSE DES SIGNAUX CAPTEURS
# =====

def analyse_signaux(df, output_dir='outputs'):
    """
    Trace l'évolution temporelle de chaque capteur analogique
    avec mise en évidence des zones de pannes.
    """
    print("\n" + "=" * 65)
    print("  DIAGNOSTIC — Analyse des signaux capteurs")
    print("=" * 65)

    os.makedirs(output_dir, exist_ok=True)
    df_s = df.iloc[:, :100].copy()

    capteurs_analogiques = [c for c in FEATURE_COLS if c in df.columns]
    n = len(capteurs_analogiques)
    cols_plot = 2
    rows_plot = (n + 1) // cols_plot

    fig, axes = plt.subplots(rows_plot, cols_plot, figsize=(18, rows_plot * 3.5))
    fig.suptitle("Analyse des signaux capteurs — MetroPT-3", fontsize=14, fontweight='bold')
    axes = axes.flatten()

    couleurs = ['#3498db', '#2ecc71', '#e74c3c', '#9b59b6', '#e67e22', '#1abc9c', '#e91e63']

    for i, col in enumerate(capteurs_analogiques):
        ax = axes[i]
        ax.plot(df_s['timestamp'], df_s[col],
               color=couleurs[i % len(couleurs)], linewidth=0.6, alpha=0.85)

    # Zones de pannes
    premier = True
    for debut_str, fin_str in FAILURE_WINDOWS:
        lbl = 'Panne' if premier else "
        ax.axvspan(pd.to_datetime(debut_str), pd.to_datetime(fin_str),
                  alpha=0.25, color='red', label=lbl)
        premier = False

```

```

    ax.set_title(col, fontsize=11, fontweight='bold')
    ax.set_xlabel('Date', fontsize=9)
    ax.xaxis.set_major_formatter(mdates.DateFormatter('%b %Y'))
    ax.xaxis.set_major_locator(mdates.MonthLocator())
    plt.setp(ax.xaxis.get_majorticklabels(), rotation=20)
    ax.grid(True, alpha=0.3)
    ax.set_facecolor('#fdfdfd')
    if premier is False:
        ax.legend(fontsize=8, loc='upper right')

# Masquer axes inutilisés
for j in range(len(capteurs_analogiques), len(axes)):
    axes[j].set_visible(False)

plt.tight_layout()
chemin = os.path.join(output_dir, 'diagnostic_signaux.png')
plt.savefig(chemin, dpi=140, bbox_inches='tight')
plt.show()
print(f" ✓ Graphique sauvegardé : {chemin}")

# =====
# 5. VISUALISATION DES RÉSULTATS DE CLASSIFICATION
# =====

def plot_diagnostic(resultats, y_test, output_dir='outputs'):
    """
    Trace les matrices de confusion et la comparaison des modèles.
    """
    os.makedirs(output_dir, exist_ok=True)

    noms = list(resultats.keys())
    n_mod = len(noms)

    fig, axes = plt.subplots(2, n_mod, figsize=(6 * n_mod, 10))
    fig.suptitle("Diagnostic — Résultats des modèles de classification",
                 fontsize=14, fontweight='bold')

    for i, nom in enumerate(noms):
        res = resultats[nom]

        # — Matrice de confusion —
        ax_cm = axes[0, i]
        sns.heatmap(
            res['cm'], annot=True, fmt='d', cmap='Blues',
            xticklabels=['Normal', 'Panne'],
            yticklabels=['Normal', 'Panne'],
            ax=ax_cm, cbar=False
        )
        ax_cm.set_title(f"{nom}\nAcc={res['acc']:.4f} | F1={res['f1']:.4f}",
                       fontsize=10, fontweight='bold')
        ax_cm.set_xlabel('Prédit', fontsize=9)
        ax_cm.set_ylabel('Réal', fontsize=9)

        # — Barres accuracy / F1 —
        ax_bar = axes[1, i]

```

```

    metriques = ['Accuracy', 'F1-score']
    valeurs = [res['acc'], res['f1']]
    bars = ax_bar.bar(metriques, valeurs,
                      color=['#3498db', '#2ecc71'], edgecolor='white', width=0.4)
    for bar, val in zip(bars, valeurs):
        ax_bar.text(bar.get_x() + bar.get_width() / 2,
                    bar.get_height() + 0.005,
                    f'{val:.4f}', ha='center', va='bottom',
                    fontsize=11, fontweight='bold')
    ax_bar.set_ylim(0, 1.1)
    ax_bar.set_facecolor('#fdfdfd')
    ax_bar.grid(True, axis='y', alpha=0.4)

plt.tight_layout()
chemin = os.path.join(output_dir, 'diagnostic_modeles.png')
plt.savefig(chemin, dpi=140, bbox_inches='tight')
plt.show()
print(f" ✓ Graphique sauvegardé : {chemin}")

# — Importance des features (Random Forest) —
if 'Random Forest' in resultats:
    print("\n Importance des features (Random Forest) :")
    rf = resultats['Random Forest']['modele']
    if hasattr(rf, 'feature_importances_'):
        importances = rf.feature_importances_
        cols_dispo = [c for c in FEATURE_COLS]
        if len(importances) == len(cols_dispo):
            imp_df = pd.DataFrame({
                'feature' : cols_dispo,
                'importance': importances
            }).sort_values('importance', ascending=False)

            print(imp_df.to_string(index=False))

            fig2, ax = plt.subplots(figsize=(9, 4))
            ax.barh(imp_df['feature'], imp_df['importance'],
                   color='#3498db', edgecolor='white')
            ax.set_title("Importance des features — Random Forest",
                        fontsize=12, fontweight='bold')
            ax.set_xlabel("Importance")
            ax.invert_yaxis()
            ax.set_facecolor('#fdfdfd')
            plt.tight_layout()
            chemin2 = os.path.join(output_dir, 'diagnostic_feature_importance.png')
            plt.savefig(chemin2, dpi=140, bbox_inches='tight')
            plt.show()
            print(f" ✓ Graphique sauvegardé : {chemin2}")

```

A.4 pronostic.py → Estimation du RUL

```

=====

```

```

pronostic.py
Section 5 — Maintenance Prédictive : PRONOSTIC

```

```

=====
- Estimation de la durée de vie résiduelle (RUL)

```

- Modèles : Régression linéaire, Random Forest, LSTM
- Comparaison des résultats

```
=====
"""

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
import os
import warnings
warnings.filterwarnings('ignore')

from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.preprocessing import StandardScaler, MinMaxScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

# Pannes officielles (dates de fin = événements de défaillance)
FAILURE_DATES = [
    pd.Timestamp('2020-04-18 23:59:00'),
    pd.Timestamp('2020-05-30 06:00:00'),
    pd.Timestamp('2020-06-07 14:30:00'), # fin maintenance panne #3
    pd.Timestamp('2020-07-16 00:00:00'), # fin maintenance panne #4
]

FEATURE_COLS = [
    'TP2', 'TP3', 'H1', 'DV_pressure', 'Reservoirs',
    'Oil_temperature', 'Motor_current'
]

# =====
# 1. CALCUL DU RUL (Remaining Useful Life)
# =====

def compute_rul(df):
    """
    Calcule le RUL pour chaque point de données.
    RUL = temps restant avant la prochaine panne (en heures).

    Stratégie : pour chaque timestamp, on cherche
    la prochaine date de panne et on calcule la différence.
    """
    print("\n" + "=" * 65)
    print(" PRONOSTIC — Calcul du RUL (Remaining Useful Life)")
    print("=" * 65)

    df = df.copy()
    df['RUL'] = np.nan

    for i, ts in enumerate(df['timestamp']):
        # Trouver la prochaine panne après ce timestamp
        futures = [d for d in FAILURE_DATES if d > ts]
```

```

    if futures:
        prochaine_panne = min(futures)
        df.at[i, 'RUL'] = (prochaine_panne - ts).total_seconds() / 3600

# Supprimer les lignes sans RUL (après la dernière panne)
df = df.dropna(subset=['RUL']).reset_index(drop=True)

print(f" Lignes avec RUL calculé : {len(df):,}")
print(f" RUL min   : {df['RUL'].min():.2f} h")
print(f" RUL max   : {df['RUL'].max():.2f} h")
print(f" RUL moyen : {df['RUL'].mean():.2f} h")

return df

# =====
# 2. PRÉPARATION DES FEATURES POUR LE PRONOSTIC
# =====

def prepare_rul_data(df, sample_rate=20):
    """
    Prépare X, y pour l'estimation du RUL.
    Ajoute des features temporelles (rolling stats).
    """
    df_s = df.iloc[:, :sample_rate].copy().reset_index(drop=True)
    cols = [c for c in FEATURE_COLS if c in df_s.columns]

    # Features de base
    X = df_s[cols].fillna(df_s[cols].median())

    # Features supplémentaires : moyennes glissantes (fenêtre 10)
    for col in cols:
        X[f'{col}_roll_mean'] = X[col].rolling(10, min_periods=1).mean()
        X[f'{col}_roll_std'] = X[col].rolling(10, min_periods=1).std().fillna(0)

    y = df_s['RUL']

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42
    )

    scaler = StandardScaler()
    X_train = scaler.fit_transform(X_train)
    X_test = scaler.transform(X_test)

    print(f"\n Features pour RUL : {X.shape[1]}")
    print(f" Train : {len(X_train):,} | Test : {len(X_test):,}")

    return X_train, X_test, y_train, y_test, scaler

# =====
# 3. MODÈLES DE RÉGRESSION
# =====

def train_rul_models(X_train, X_test, y_train, y_test):

```

```
#####
Entraîne Régression linéaire et Random Forest Regressor.
#####

print("\n" + "=" * 65)
print(" PRONOSTIC — Entraînement des modèles RUL")
print("=" * 65)

modeles = {
    'Régression linéaire' : LinearRegression(),
    'Random Forest'      : RandomForestRegressor(
        n_estimators=100, max_depth=10,
        random_state=42, n_jobs=-1),
}

resultats = {}
for nom, modele in modeles.items():
    print(f"\n ► {nom} ...")
    modele.fit(X_train, y_train)
    y_pred = modele.predict(X_test)
    y_pred = np.maximum(y_pred, 0) # RUL ≥ 0

    rmse = np.sqrt(mean_squared_error(y_test, y_pred))
    mae = mean_absolute_error(y_test, y_pred)
    r2 = r2_score(y_test, y_pred)

    resultats[nom] = {
        'modele': modele,
        'y_pred': y_pred,
        'rmse' : rmse,
        'mae' : mae,
        'r2' : r2,
    }
    print(f" RMSE : {rmse:.4f} h")
    print(f" MAE : {mae:.4f} h")
    print(f" R² : {r2:.4f}")

return resultats

# =====
# 4. MODÈLE LSTM (Réseau de neurones — séries temporelles)
# =====

def train_lstm_rul(df, output_dir='outputs', sample_rate=50, seq_len=30):
    #####
    Entraîne un LSTM pour l'estimation du RUL.
    Utilise TensorFlow/Keras si disponible, sinon affiche un message.
    #####

    print("\n" + "=" * 65)
    print(" PRONOSTIC — Modèle LSTM (réseau de neurones)")
    print("=" * 65)

    try:
        import tensorflow as tf
        from tensorflow.keras.models import Sequential
        from tensorflow.keras.layers import LSTM, Dense, Dropout
```

```

from tensorflow.keras.callbacks import EarlyStopping
print(f" TensorFlow version : {tf.__version__}")
except ImportError:
    print(" ⚠ TensorFlow non installé. LSTM ignoré.")
    print("    Pour l'activer : pip install tensorflow")
    return None, None, None

cols = [c for c in FEATURE_COLS if c in df.columns]
df_s = df.iloc[:,sample_rate].copy().reset_index(drop=True)
df_s = df_s.dropna(subset=['RUL'])

# Normalisation
scaler_X = MinMaxScaler()
scaler_y = MinMaxScaler()
X_scaled = scaler_X.fit_transform(df_s[cols].fillna(0))
y_scaled = scaler_y.fit_transform(df_s[['RUL']])

# Construction des séquences
def make_sequences(X, y, seq_len):
    Xs, ys = [], []
    for i in range(len(X) - seq_len):
        Xs.append(X[i:i + seq_len])
        ys.append(y[i + seq_len])
    return np.array(Xs), np.array(ys)

X_seq, y_seq = make_sequences(X_scaled, y_scaled, seq_len)
split = int(len(X_seq) * 0.8)
X_train, X_test = X_seq[:split], X_seq[split:]
y_train, y_test = y_seq[:split], y_seq[split:]

print(f" Séquences — Train: {len(X_train):,} Test: {len(X_test):,}")
print(f" Shape entrée LSTM : {X_train.shape}")

# Architecture LSTM
model = Sequential([
    LSTM(64, input_shape=(seq_len, len(cols)), return_sequences=True),
    Dropout(0.2),
    LSTM(32),
    Dropout(0.2),
    Dense(16, activation='relu'),
    Dense(1)
])
model.compile(optimizer='adam', loss='mse', metrics=['mae'])
model.summary()

# Entraînement
es = EarlyStopping(patience=5, restore_best_weights=True)
history = model.fit(
    X_train, y_train,
    epochs=30, batch_size=64,
    validation_split=0.1,
    callbacks=[es], verbose=1
)

# Évaluation
y_pred_scaled = model.predict(X_test)

```

```

y_pred = scaler_y.inverse_transform(y_pred_scaled).flatten()
y_real = scaler_y.inverse_transform(y_test).flatten()
y_pred = np.maximum(y_pred, 0)

rmse = np.sqrt(mean_squared_error(y_real, y_pred))
mae = mean_absolute_error(y_real, y_pred)
r2 = r2_score(y_real, y_pred)

print(f"\n LSTM — RMSE : {rmse:.4f} h | MAE : {mae:.4f} h | R² : {r2:.4f}")

return model, history, {'rmse': rmse, 'mae': mae, 'r2': r2,
                        'y_pred': y_pred, 'y_real': y_real}

# =====
# 5. VISUALISATION DES RÉSULTATS RUL
# =====

def plot_pronostic(df_rul, resultats_reg, lstm_resultats=None, output_dir='outputs'):
    """
    Visualise les résultats d'estimation du RUL.
    """
    os.makedirs(output_dir, exist_ok=True)

    noms = list(resultats_reg.keys())
    n_fig = len(noms) + (1 if lstm_resultats else 0)

    fig, axes = plt.subplots(2, max(2, n_fig), figsize=(7 * max(2, n_fig), 12))
    fig.suptitle("Pronostic — Estimation du RUL (Remaining Useful Life)",
                 fontsize=14, fontweight='bold')

    # — Ligne 1 : RUL prédit vs réel —
    for i, nom in enumerate(noms):
        ax = axes[0, i]
        res = resultats_reg[nom]
        y_test_arr = np.array(res.get('y_test', []))
        y_pred_arr = res['y_pred']
        n_plot = min(500, len(y_pred_arr))

        ax.scatter(range(n_plot), y_pred_arr[:n_plot],
                    s=4, alpha=0.5, color='#e74c3c', label='RUL prédit')
        if len(y_test_arr) > 0:
            ax.scatter(range(n_plot), np.array(y_test_arr)[:n_plot],
                        s=4, alpha=0.3, color='#3498db', label='RUL réel')
        ax.set_title(f'{nom}\nRMSE={res['rmse']:.2f}h | R²={res['r2']:.4f}',
                     fontsize=10, fontweight='bold')
        ax.set_xlabel("Échantillons")
        ax.set_ylabel("RUL (heures)")
        ax.legend(fontsize=8)
        ax.set_facecolor('#fdfdfd')

    # — LSTM si disponible —
    if lstm_resultats:
        ax = axes[0, len(noms)]
        n_plot = min(500, len(lstm_resultats['y_pred']))
        ax.scatter(range(n_plot), lstm_resultats['y_pred'][:n_plot],

```

```

        s=4, alpha=0.5, color='#e74c3c', label='RUL prédit')
    ax.scatter(range(n_plot), lstm_resultats['y_real'][:n_plot],
               s=4, alpha=0.3, color='#3498db', label='RUL réel')
    ax.set_title(f'LSTM\nRMSE={lstm_resultats['rmse']:.2f}h | R²={lstm_resultats['r2']:.4f}',
                 fontsize=10, fontweight='bold')
    ax.set_xlabel('Échantillons')
    ax.set_ylabel('RUL (heures)')
    ax.legend(fontsize=8)
    ax.set_facecolor('#fdfdfd')

# — Ligne 2 : Timeline RUL réel —
ax_rul = axes[1, 0]
df_s = df_rul.iloc[:, :50]
ax_rul.plot(df_s['timestamp'], df_s['RUL'],
            color='#3498db', linewidth=0.7, alpha=0.85, label='RUL réel (h)')
for fd in FAILURE_DATES:
    ax_rul.axvline(x=fd, color='red', linestyle='--', linewidth=1.5, alpha=0.7)
ax_rul.set_title('Évolution temporelle du RUL', fontsize=11, fontweight='bold')
ax_rul.set_xlabel('Date')
ax_rul.set_ylabel('RUL (heures)')
ax_rul.xaxis.set_major_formatter(mdates.DateFormatter('%b %Y'))
ax_rul.legend(fontsize=9)
ax_rul.set_facecolor('#fdfdfd')

# — Comparaison RMSE —
ax_comp = axes[1, 1]
tous_noms = list(noms) + (['LSTM'] if lstm_resultats else [])
tous_rmse = [resultats_reg[n]['rmse'] for n in noms]
tous_r2 = [resultats_reg[n]['r2'] for n in noms]
if lstm_resultats:
    tous_rmse.append(lstm_resultats['rmse'])
    tous_r2.append(lstm_resultats['r2'])

x = np.arange(len(tous_noms))
w = 0.35
bars1 = ax_comp.bar(x - w/2, tous_rmse, w, label='RMSE (h)', color='#e74c3c', alpha=0.8)
ax_comp2 = ax_comp.twinx()
bars2 = ax_comp2.bar(x + w/2, tous_r2, w, label='R²', color='#2ecc71', alpha=0.8)

ax_comp.set_xticks(x)
ax_comp.set_xticklabels(tous_noms, rotation=10, fontsize=9)
ax_comp.set_ylabel('RMSE (heures)', color='#e74c3c')
ax_comp2.set_ylabel('R²', color='#2ecc71')
ax_comp.set_title('Comparaison des modèles RUL', fontsize=11, fontweight='bold')
ax_comp.set_facecolor('#fdfdfd')

lines1, labels1 = ax_comp.get_legend_handles_labels()
lines2, labels2 = ax_comp2.get_legend_handles_labels()
ax_comp.legend(lines1 + lines2, labels1 + labels2, fontsize=9)

# Masquer axes restants
for j in range(len(noms) + (1 if lstm_resultats else 0), axes.shape[1]):
    if j < axes.shape[1]:
        axes[0, j].set_visible(False)
for j in range(2, axes.shape[1]):
    axes[1, j].set_visible(False)

```

```
plt.tight_layout()
chemin = os.path.join(output_dir, 'pronostic_rul.png')
plt.savefig(chemin, dpi=140, bbox_inches='tight')
plt.show()
print(f" ✓ Graphique sauvegardé : {chemin}")
```

A.5 validation.py → Métriques et courbes de validation

```
"""
=====
validation.py
Section 6 — Validation des modèles
=====
- Métriques : précision, rappel, F1, RMSE, score RUL
- Courbes ROC, courbes d'apprentissage
- Matrices de confusion
- Comparaison globale des modèles
=====
"""

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
import seaborn as sns
import os
import warnings
warnings.filterwarnings('ignore')

from sklearn.metrics import (
    confusion_matrix, classification_report,
    roc_curve, auc, precision_recall_curve,
    accuracy_score, f1_score, precision_score, recall_score
)
from sklearn.model_selection import learning_curve

# =====
# 1. MÉTRIQUES DÉTAILLÉES — CLASSIFICATION
# =====

def evaluate_classification(resultats_classif, X_test, y_test, output_dir='outputs'):
    """
    Calcule et affiche les métriques complètes pour chaque
    modèle de classification (SVM, Arbre, Random Forest).

    Métriques : Accuracy, Précision, Rappel, F1, AUC-ROC
    """
    print("\n" + "=" * 65)
    print(" VALIDATION — Métriques de classification")
    print("=" * 65)

    os.makedirs(output_dir, exist_ok=True)
```

```
# Tableau récapitulatif
tableau = []
for nom, res in resultats_classif.items():
    y_pred = res['y_pred']
    row = {
        'Modèle' : nom,
        'Accuracy' : round(accuracy_score(y_test, y_pred), 4),
        'Précision' : round(precision_score(y_test, y_pred, zero_division=0), 4),
        'Rappel' : round(recall_score(y_test, y_pred, zero_division=0), 4),
        'F1-score' : round(f1_score(y_test, y_pred, zero_division=0), 4),
    }
    # AUC-ROC si predict_proba disponible
    modele = res['modele']
    if hasattr(modele, 'predict_proba'):
        from sklearn.metrics import roc_auc_score
        y_proba = modele.predict_proba(X_test)[: , 1]
        row['AUC-ROC'] = round(roc_auc_score(y_test, y_proba), 4)
    else:
        row['AUC-ROC'] = 'N/A'
    tableau.append(row)

df_metriques = pd.DataFrame(tableau)
print(f"\n{df_metriques.to_string(index=False)}")

return df_metriques
```

```
# =====
# 2. MÉTRIQUES DÉTAILLÉES — RÉGRESSION (RUL)
# =====
```

```
def evaluate_regression(resultats_reg, lstm_res=None, output_dir='outputs'):
    """
```

Calcule RMSE, MAE, R² et score RUL NASA pour chaque modèle de pronostic.

```
    """
    print("\n" + "=" * 65)
    print(" VALIDATION — Métriques de régression (RUL)")
    print("=" * 65)
```

```
def score_rul_nasa(y_true, y_pred):
    """
```

Score NASA pour RUL :
 Pénalise davantage les prédictions trop tardives (sous-estimation).
 $S = \sum \exp(-d/13) - 1$ si $d < 0$ (sur-estimation)
 $S = \sum \exp(d/10) - 1$ si $d \geq 0$ (sous-estimation)
 où $d = y_{\text{pred}} - y_{\text{true}}$

```
    """
    d = np.array(y_pred) - np.array(y_true)
    score = np.where(d < 0,
                    np.exp(-d / 13) - 1,
                    np.exp(d / 10) - 1)
    return round(np.sum(score), 4)
```

```
tableau = []
for nom, res in resultats_reg.items():
```

```

from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
y_pred = res['y_pred']
y_true = res.get('y_true', res.get('y_test', []))
if len(y_true) == 0:
    continue
row = {
    'Modèle' : nom,
    'RMSE (h)' : round(np.sqrt(mean_squared_error(y_true, y_pred)), 4),
    'MAE (h)' : round(mean_absolute_error(y_true, y_pred), 4),
    'R²' : round(r2_score(y_true, y_pred), 4),
    'Score RUL' : score_rul_nasa(y_true, y_pred),
}
tableau.append(row)

```

```

if lstm_res:
    from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
    y_pred = lstm_res['y_pred']
    y_true = lstm_res['y_real']
    tableau.append({
        'Modèle' : 'LSTM',
        'RMSE (h)' : round(np.sqrt(mean_squared_error(y_true, y_pred)), 4),
        'MAE (h)' : round(mean_absolute_error(y_true, y_pred), 4),
        'R²' : round(r2_score(y_true, y_pred), 4),
        'Score RUL' : score_rul_nasa(y_true, y_pred),
    })

```

```

if tableau:
    df_reg = pd.DataFrame(tableau)
    print(f"\n{df_reg.to_string(index=False)}")
    return df_reg
return None

```

```

# =====
# 3. COURBES ROC
# =====

```

```

def plot_roc_curves(resultats_classif, X_test, y_test, output_dir='outputs'):

```

```

    """
    Trace les courbes ROC pour les modèles de classification.
    """

```

```

    os.makedirs(output_dir, exist_ok=True)

```

```

    fig, ax = plt.subplots(figsize=(8, 6))
    ax.set_facecolor('#fdfdfd')
    couleurs = ['#3498db', '#e74c3c', '#2ecc71', '#9b59b6']

```

```

    for i, (nom, res) in enumerate(resultats_classif.items()):
        modele = res['modele']
        if hasattr(modele, 'predict_proba'):
            y_proba = modele.predict_proba(X_test)[: , 1]
            fpr, tpr, _ = roc_curve(y_test, y_proba)
            roc_auc = auc(fpr, tpr)
            ax.plot(fpr, tpr, color=couleurs[i % len(couleurs)],
                    linewidth=2, label=f"{nom} (AUC = {roc_auc:.4f})")
        elif hasattr(modele, 'decision_function'):

```

```

y_score = modele.decision_function(X_test)
fpr, tpr, _ = roc_curve(y_test, y_score)
roc_auc = auc(fpr, tpr)
ax.plot(fpr, tpr, color=couleurs[i % len(couleurs)],
        linewidth=2, label=f'{nom} (AUC = {roc_auc:.4f})')

ax.plot([0, 1], [0, 1], 'k--', linewidth=1, label='Aléatoire (AUC = 0.5)')
ax.set_title('Courbes ROC — Modèles de classification', fontsize=13, fontweight='bold')
ax.set_xlabel('Taux de faux positifs (FPR)', fontsize=11)
ax.set_ylabel('Taux de vrais positifs (TPR)', fontsize=11)
ax.legend(fontsize=10)
ax.grid(True, alpha=0.3)

plt.tight_layout()
chemin = os.path.join(output_dir, 'validation_roc.png')
plt.savefig(chemin, dpi=140, bbox_inches='tight')
plt.show()
print(f" ✓ Courbes ROC sauvegardées : {chemin}")

```

```

# =====
# 4. COURBES D'APPRENTISSAGE
# =====

```

```

def plot_learning_curves(resultats_classif, X_train, y_train, output_dir='outputs'):
    """

```

```

    Trace les courbes d'apprentissage (train vs validation score)
    pour détecter sur-apprentissage ou sous-apprentissage.
    """

```

```

    os.makedirs(output_dir, exist_ok=True)

```

```

    noms = list(resultats_classif.keys())
    fig, axes = plt.subplots(1, len(noms), figsize=(7 * len(noms), 5))
    if len(noms) == 1:
        axes = [axes]

```

```

    for i, nom in enumerate(noms):
        modele = resultats_classif[nom]['modele']
        ax = axes[i]

```

```

        try:
            train_sizes, train_scores, val_scores = learning_curve(
                modele, X_train, y_train,
                cv=3, n_jobs=-1,
                train_sizes=np.linspace(0.2, 1.0, 6),
                scoring='f1_weighted'
            )
            train_mean = train_scores.mean(axis=1)
            val_mean = val_scores.mean(axis=1)
            train_std = train_scores.std(axis=1)
            val_std = val_scores.std(axis=1)

```

```

            ax.plot(train_sizes, train_mean, 'o-', color='#3498db',
                    linewidth=2, label='Train')
            ax.fill_between(train_sizes,
                           train_mean - train_std,

```

```

        train_mean + train_std, alpha=0.15, color='#3498db')

    ax.plot(train_sizes, val_mean, 'o-', color='#e74c3c',
            linewidth=2, label='Validation')
    ax.fill_between(train_sizes,
                    val_mean - val_std,
                    val_mean + val_std, alpha=0.15, color='#e74c3c')

    ax.set_title(f'Courbe d'apprentissage\n{nom}', fontsize=11, fontweight='bold')
    ax.set_xlabel("Taille d'entraînement", fontsize=10)
    ax.set_ylabel('F1-score', fontsize=10)
    ax.legend(fontsize=10)
    ax.set_ylim(0, 1.1)
    ax.grid(True, alpha=0.3)
    ax.set_facecolor('#fdfdfd')
except Exception as e:
    ax.text(0.5, 0.5, f'Erreur : {str(e)[:60}}',
           ha='center', va='center', transform=ax.transAxes, fontsize=9)
    ax.axis('off')

plt.tight_layout()
chemin = os.path.join(output_dir, 'validation_learning_curves.png')
plt.savefig(chemin, dpi=140, bbox_inches='tight')
plt.show()
print(f" ✓ Courbes d'apprentissage sauvegardées : {chemin}")

# =====
# 5. TABLEAU DE BORD GLOBAL DE VALIDATION
# =====

def plot_validation_dashboard(df_metriques_classif, df_metriques_reg, output_dir='outputs'):
    """
    Tableau de bord récapitulatif avec toutes les métriques.
    """
    os.makedirs(output_dir, exist_ok=True)

    fig = plt.figure(figsize=(18, 10))
    fig.suptitle("Tableau de bord — Validation des modèles",
                 fontsize=15, fontweight='bold')

    # — Métriques classification (heatmap) —
    ax1 = fig.add_subplot(1, 2, 1)
    if df_metriques_classif is not None and not df_metriques_classif.empty:
        cols_num = ['Accuracy', 'Précision', 'Rappel', 'F1-score']
        cols_dispo = [c for c in cols_num if c in df_metriques_classif.columns]
        mat = df_metriques_classif.set_index('Modèle')[cols_dispo].astype(float)
        sns.heatmap(mat, annot=True, fmt='.4f', cmap='RdYlGn',
                    vmin=0, vmax=1, ax=ax1, cbar=True,
                    linewidths=0.5, linecolor='white')
        ax1.set_title('Métriques — Classification\n(0=mauvais → 1=parfait)',
                     fontsize=12, fontweight='bold')
        ax1.set_ylabel("")
    else:
        ax1.text(0.5, 0.5, "Pas de données classification",
                ha='center', va='center', transform=ax1.transAxes)

```

```

ax1.axis('off')

# — Métriques régression RUL —
ax2 = fig.add_subplot(1, 2, 2)
if df_metriques_reg is not None and not df_metriques_reg.empty:
    noms_reg = df_metriques_reg['Modèle'].tolist()
    rmse_vals = df_metriques_reg['RMSE (h)'].tolist()
    r2_vals = df_metriques_reg['R²'].tolist()

    x = np.arange(len(noms_reg))
    w = 0.35
    ax2b = ax2.twinx()

    bars1 = ax2.bar(x - w/2, rmse_vals, w, color='#e74c3c',
                    alpha=0.8, label='RMSE (h)', edgecolor='white')
    bars2 = ax2b.bar(x + w/2, r2_vals, w, color='#2ecc71',
                    alpha=0.8, label='R²', edgecolor='white')

    for bar, val in zip(bars1, rmse_vals):
        ax2.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.5,
                 f'{val:.2f}', ha='center', va='bottom', fontsize=9, fontweight='bold')
    for bar, val in zip(bars2, r2_vals):
        ax2b.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.01,
                  f'{val:.4f}', ha='center', va='bottom', fontsize=9, fontweight='bold')

    ax2.set_xticks(x)
    ax2.set_xticklabels(noms_reg, rotation=10, fontsize=10)
    ax2.set_ylabel('RMSE (heures)', color='#e74c3c', fontsize=10)
    ax2b.set_ylabel('R²', color='#2ecc71', fontsize=10)
    ax2.set_title('Métriques — Régression RUL', fontsize=12, fontweight='bold')
    ax2.set_facecolor('#fdfdfd')

    lines1, labels1 = ax2.get_legend_handles_labels()
    lines2, labels2 = ax2b.get_legend_handles_labels()
    ax2.legend(lines1 + lines2, labels1 + labels2, fontsize=9)
else:
    ax2.text(0.5, 0.5, "Pas de données régression",
            ha='center', va='center', transform=ax2.transAxes)
    ax2.axis('off')

plt.tight_layout()
chemin = os.path.join(output_dir, 'validation_dashboard.png')
plt.savefig(chemin, dpi=140, bbox_inches='tight')
plt.show()
print(f" ✓ Tableau de bord validation sauvegardé : {chemin}")

```

A.6 synthese.py → Tableau de bord synthèse générale

```

=====
synthese.py
Section 7 — Synthèse et Recommandations
=====

```

- Discussion sur les résultats obtenus
- Limites des approches utilisées
- Perspectives d'amélioration
- Rapport final (console + figure)

```
=====
"""

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import os

# =====
# 1. GÉNÉRATION DU RAPPORT TEXTUEL COMPLET
# =====

def generer_rapport(resultats_fmfs, df_metriques_classif=None,
                    df_metriques_reg=None, output_dir='outputs'):
    """
    Génère un rapport complet en console et dans un fichier .txt.
    """
    os.makedirs(output_dir, exist_ok=True)

    lignes = []
    sep = "=" * 65

    def a(texte=""):
        lignes.append(texte)
        print(texte)

    a(sep)
    a(" SECTION 7 — SYNTHÈSE ET RECOMMANDATIONS")
    a(" MetroPT-3 Air Compressor — Porto (2020)")
    a(sep)

    # -----
    # 7.1 Discussion sur les résultats FMFS
    # -----
    a()
    a(" 7.1 — DISCUSSION SUR LES RÉSULTATS OBTENUS")
    a(" " + "-" * 60)

    mtbf = resultats_fmfs.get('MTBF', 0)
    mttr = resultats_fmfs.get('MTTR', 0)
    dispo = resultats_fmfs.get('disponibilite_fmfs', 0)
    nb = resultats_fmfs.get('nb_pannes', 0)

    a(f" > {nb} pannes de type 'Air Leak' détectées sur la période.")
    a(f" > MTBF = {mtbf:.2f} h ({mtbf/24:.1f} jours)")
    a(f" Le compresseur tombe en panne en moyenne toutes les {mtbf/24:.0f} journées")
    a(f" de fonctionnement — indicateur de fiabilité à surveiller.")

    a(f"\n > MTTR = {mttr:.2f} h")
    a(f" Le temps de réparation varie fortement ({resultats_fmfs.get('TTR_list', [])})")
```

```

a(f" La panne #3 (52.5 h) révèle une réparation particulièrement longue.")

if dispo >= 99:
    interp = "EXCELLENT — le système répond aux standards industriels (≥ 99%).\"
elif dispo >= 95:
    interp = "BON — acceptable pour un usage industriel, mais améliorable.\"
else:
    interp = "INSUFFISANT — des actions correctives sont nécessaires.\"

a(f\"\\n  > Disponibilité = {dispo:.4f}% — {interp}\")

# Métriques classification
if df_metriques_classif is not None and not df_metriques_classif.empty:
    a()
    a(\" > Résultats de classification (détection de pannes) :\")
    meilleur = df_metriques_classif.loc[df_metriques_classif['F1-score'].idxmax()]
    a(f\" Meilleur modèle : {meilleur['Modèle']}\")
    a(f\" Accuracy : {meilleur['Accuracy']:.4f} | F1-score : {meilleur['F1-score']:.4f}\")

# Métriques régression
if df_metriques_reg is not None and not df_metriques_reg.empty:
    a()
    a(\" > Résultats d'estimation du RUL :\")
    meilleur_r = df_metriques_reg.loc[df_metriques_reg['R²'].idxmax()]
    a(f\" Meilleur modèle : {meilleur_r['Modèle']}\")
    a(f\" RMSE : {meilleur_r['RMSE (h)']:.4f} h | R² : {meilleur_r['R²']:.4f}\")

# -----
# 7.2 Limites des approches
# -----
a()
a(\" 7.2 — LIMITES DES APPROCHES UTILISÉES\")
a(\" \" + \"-\" * 60)

limites = [
    (\"Dataset non labellisé\",
     \"Les labels sont déduits des rapports manuels UCI.\\n\"
     \" L'annotation automatique reste imprécise (granularité 1Hz).\"),
    (\"Déséquilibre des classes\",
     \"Les périodes de panne (~87 h) sont très minoritaires\\n\"
     \" face aux ~4000 h de fonctionnement normal → biais possible.\"),
    (\"Pannes homogènes\",
     \"Toutes les pannes sont de type 'Air Leak'.\\n\"
     \" La généralisation à d'autres types de défauts est limitée.\"),
    (\"Fenêtre temporelle courte\",
     \"6 mois de données = 4 pannes seulement.\\n\"
     \" Les modèles statistiques manquent de robustesse.\"),
    (\"RUL simplifié\",
     \"Le RUL calculé suppose une dégradation linéaire.\\n\"
     \" En réalité, la dégradation peut être non linéaire.\"),
    (\"LSTM sans données suffisantes\",
     \"Un LSTM nécessite idéalement des milliers de cycles.\\n\"
     \" Avec 4 pannes, les résultats peuvent être peu fiables.\"),
]

for titre, desc in limites:

```

```
a(f"\n ▲ {titre}")
a(f" {desc}")
```

```
# -----
# 7.3 Perspectives d'amélioration
# -----
```

```
a()
a(" 7.3 — PERSPECTIVES D'AMÉLIORATION")
a(" " + "-" * 60)
```

```
perspectives = [
    ("Court terme",
     [
         "Enrichir le dataset avec d'autres années de données.",
         "Appliquer SMOTE (oversampling) pour équilibrer les classes.",
         "Tester XGBoost et LightGBM pour la classification.",
         "Affiner la détection de pannes par analyse des résidus.",
     ]),
    ("Moyen terme",
     [
         "Implémenter un modèle LSTM bidirectionnel (BiLSTM).",
         "Utiliser des autoencodeurs pour la détection d'anomalies.",
         "Développer un tableau de bord temps réel (Streamlit/Dash).",
         "Intégrer les données de maintenance préventive.",
     ]),
    ("Long terme",
     [
         "Déploiement d'un système IoT de surveillance en continu.",
         "Modèle de jumeau numérique (Digital Twin) du compresseur.",
         "Optimisation de la planification de maintenance (CMMS).",
         "Extension à d'autres équipements du réseau de métro.",
     ]),
]
```

```
for horizon, actions in perspectives:
```

```
    a(f"\n ☞ {horizon} :")
    for action in actions:
        a(f" • {action}")
```

```
# -----
# Conclusion
# -----
```

```
a()
a(" " + "=" * 60)
a(" CONCLUSION")
a(" " + "=" * 60)
a()
a(" Cette étude a démontré l'applicabilité des méthodes FMDS")
a(" et de l'apprentissage automatique pour la maintenance")
a(" prédictive du compresseur d'air du métro de Porto.")
a()
a(f" Le compresseur présente une disponibilité de {dispo:.4f}%,")
a(f" avec un MTBF de {mtbf:.0f} h et un MTTR de {mtrr:.1f} h.")
a()
a(" Les modèles de classification permettent de détecter les")
a(" anomalies avant qu'elles ne deviennent des pannes critiques.")
```

```

a(" L'estimation du RUL ouvre la voie à une maintenance")
a(" planifiée et économiquement optimisée.")
a()
a(sep)

# Sauvegarde dans un fichier texte
chemin_txt = os.path.join(output_dir, 'rapport_synthese.txt')
with open(chemin_txt, 'w', encoding='utf-8') as f:
    f.write('\n'.join(lignes))
print(f"\n ✓ Rapport sauvegardé : {chemin_txt}")

return lignes

# =====
# 2. FIGURE DE SYNTHÈSE VISUELLE
# =====

def plot_synthese(resultats_fmfs, df_metriques_classif=None,
                  df_metriques_reg=None, output_dir='outputs'):
    """
    Génère une figure de synthèse visuelle récapitulative.
    """
    os.makedirs(output_dir, exist_ok=True)

    fig = plt.figure(figsize=(20, 12))
    fig.patch.set_facecolor('#f8f9fa')
    fig.suptitle(
        "Synthèse Générale — Projet Maintenance Prédictive MetroPT-3",
        fontsize=16, fontweight='bold', y=0.98
    )

    # — 1. KPI FMFS (texte centré) —
    ax1 = fig.add_subplot(2, 4, 1)
    ax1.axis('off')
    kpis = [
        ("MTBF", f"{resultats_fmfs.get('MTBF', 0):.1f} h", '#3498db'),
        ("MTTR", f"{resultats_fmfs.get('MTTR', 0):.1f} h", '#e67e22'),
        ("λ", f"{resultats_fmfs.get('lambda', 0):.2e}", '#e74c3c'),
        ("Disponibilité", f"{resultats_fmfs.get('disponibilite_fmfs', 0):.4f}%", '#2ecc71'),
    ]
    for j, (label, val, col) in enumerate(kpis):
        y_pos = 0.85 - j * 0.22
        ax1.text(0.15, y_pos, label, fontsize=11, fontweight='bold',
                 transform=ax1.transAxes, color='#2c3e50')
        ax1.text(0.55, y_pos, val, fontsize=12, fontweight='bold',
                 transform=ax1.transAxes, color=col)
    ax1.set_title("Indicateurs FMFS", fontsize=11, fontweight='bold', pad=10)
    ax1.add_patch(plt.Rectangle((0, 0), 1, 1, fill=True,
                                color='#ecf0f1', transform=ax1.transAxes,
                                zorder=0))

    # — 2. Camembert disponibilité —
    ax2 = fig.add_subplot(2, 4, 2)
    dispo = resultats_fmfs.get('disponibilite_fmfs', 99.9)
    indispo = 100 - dispo

```

```

ax2.pie([dispo, indispo],
        labels=[f'Dispo\n{dispo:.3f}%', f'Indispo\n{indispo:.3f}%'],
        colors=['#2ecc71', '#e74c3c'],
        autopct='%1.3f%%', startangle=90,
        wedgeprops={'edgecolor': 'white', 'linewidth': 2},
        textprops={'fontsize': 9})
ax2.set_title("Disponibilité", fontsize=11, fontweight='bold')

# — 3. Comparaison TBF —
ax3 = fig.add_subplot(2, 4, 3)
tbef_list = resultats_fmfs.get('TBF_list', [])
if tbef_list:
    ax3.bar([f'TBF {i+1}→{i+2}' for i in range(len(tbef_list))],
            tbef_list, color='#3498db', edgecolor='white', alpha=0.85)
    ax3.axhline(y=resultats_fmfs.get('MTBF', 0), color='#e67e22',
                linestyle='--', linewidth=2,
                label=f'MTBF={resultats_fmfs.get('MTBF', 0):.0f}h")
    ax3.set_title("TBF entre pannes", fontsize=11, fontweight='bold')
    ax3.set_ylabel("Heures")
    ax3.legend(fontsize=9)
    ax3.set_facecolor('#fdfdfd')

# — 4. Comparaison TTR —
ax4 = fig.add_subplot(2, 4, 4)
ttr_list = resultats_fmfs.get('TTR_list', [])
if ttr_list:
    cols_ttr = ['#2ecc71' if t <= resultats_fmfs.get('MTTR', 0) else '#e74c3c'
                for t in ttr_list]
    ax4.bar([f'TTR {i+1}' for i in range(len(ttr_list))],
            ttr_list, color=cols_ttr, edgecolor='white', alpha=0.85)
    ax4.axhline(y=resultats_fmfs.get('MTTR', 0), color='#e67e22',
                linestyle='--', linewidth=2,
                label=f'MTTR={resultats_fmfs.get('MTTR', 0):.1f}h")
    ax4.set_title("TTR par panne", fontsize=11, fontweight='bold')
    ax4.set_ylabel("Heures")
    ax4.legend(fontsize=9)
    ax4.set_facecolor('#fdfdfd')

# — 5. Métriques classification —
ax5 = fig.add_subplot(2, 4, 5)
if df_metriques_classif is not None and not df_metriques_classif.empty:
    cols_m = [c for c in ['Accuracy', 'F1-score'] if c in df_metriques_classif.columns]
    modeles = df_metriques_classif['Modèle'].tolist()
    x = np.arange(len(modeles))
    w = 0.35
    for k, col_m in enumerate(cols_m):
        vals = df_metriques_classif[col_m].astype(float).tolist()
        offset = (k - len(cols_m)/2 + 0.5) * w
        bars = ax5.bar(x + offset, vals, w,
                        label=col_m,
                        color=['#3498db', '#2ecc71'][k],
                        edgecolor='white', alpha=0.85)
    ax5.set_xticks(x)
    ax5.set_xticklabels(modeles, rotation=10, fontsize=8)
    ax5.set_ylim(0, 1.1)
    ax5.set_title("Classification — Accuracy & F1", fontsize=11, fontweight='bold')

```

```

    ax5.legend(fontsize=9)
    ax5.set_facecolor('#fdfdfd')
    ax5.grid(True, axis='y', alpha=0.3)
else:
    ax5.text(0.5, 0.5, "Classification\nnon exécutée",
            ha='center', va='center', transform=ax5.transAxes, fontsize=11)
    ax5.axis('off')

# — 6. Métriques RUL —
ax6 = fig.add_subplot(2, 4, 6)
if df_metriques_reg is not None and not df_metriques_reg.empty:
    modes_r = df_metriques_reg['Modèle'].tolist()
    r2_vals = df_metriques_reg['R²'].astype(float).tolist()
    couleurs_r = ['#9b59b6', '#1abc9c', '#e91e63']
    ax6.barh(modes_r, r2_vals,
            color=couleurs_r[:len(modes_r)], edgecolor='white', alpha=0.85)
    ax6.set_xlim(0, 1.1)
    ax6.set_title("Pronostic RUL — R²", fontsize=11, fontweight='bold')
    ax6.set_xlabel("R²")
    ax6.set_facecolor('#fdfdfd')
    ax6.grid(True, axis='x', alpha=0.3)
else:
    ax6.text(0.5, 0.5, "Pronostic RUL\nnon exécuté",
            ha='center', va='center', transform=ax6.transAxes, fontsize=11)
    ax6.axis('off')

# — 7. Limites (texte) —
ax7 = fig.add_subplot(2, 4, 7)
ax7.axis('off')
ax7.add_patch(plt.Rectangle((0, 0), 1, 1, fill=True,
                           color='#fff3e0', transform=ax7.transAxes, zorder=0))
ax7.text(0.5, 0.97, "⚠️ Limites", ha='center', va='top',
        fontsize=11, fontweight='bold', transform=ax7.transAxes, color='#e67e22')
limites_txt = [
    "Dataset non labellisé",
    "4 pannes seulement",
    "Déséquilibre des classes",
    "Pannes homogènes (Air Leak)",
    "RUL linéaire simplifié",
]
for k, lim in enumerate(limites_txt):
    ax7.text(0.08, 0.82 - k * 0.15, f"• {lim}", fontsize=9,
            transform=ax7.transAxes, color='#7f8c8d')

# — 8. Perspectives (texte) —
ax8 = fig.add_subplot(2, 4, 8)
ax8.axis('off')
ax8.add_patch(plt.Rectangle((0, 0), 1, 1, fill=True,
                           color='#e8f5e9', transform=ax8.transAxes, zorder=0))
ax8.text(0.5, 0.97, "🔮 Perspectives", ha='center', va='top',
        fontsize=11, fontweight='bold', transform=ax8.transAxes, color='#27ae60')
perspectives_txt = [
    "Enrichir le dataset",
    "SMOTE pour rééquilibrer",
    "BiLSTM / Autoencodeurs",
    "Dashboard temps réel",
]

```

```
"Jumeau numérique",
]
for k, p in enumerate(perspectives_txt):
    ax8.text(0.08, 0.82 - k * 0.15, f"• {p}", fontsize=9,
            transform=ax8.transAxes, color='#2c3e50')

plt.tight_layout(rect=[0, 0, 1, 0.96])
chemin = os.path.join(output_dir, 'synthese_finale.png')
plt.savefig(chemin, dpi=140, bbox_inches='tight', facecolor='#f8f9fa')
plt.show()
print(f" ✓ Figure de synthèse sauvegardée : {chemin}")
```